RZT LYRICS

От каркаса к первому MP4

Подробный план для шести участников

2 frontend · 2 backend · 2 алгоритма
Python backend · QA совмещает Татьяна

Внутренний релиз: 27 декабря 2026
Резерв: 28-31 декабря

Внутри: точные файлы и функции, порядок реализации, входы и выходы, проверки, передача между участниками и персональные команды GitHub.

Статус: стартовый каркас. Генерация видео ещё не реализована. Документ описывает будущую работу; выполненные проверки каркаса перечислены отдельно.

Редакция 9 ноября 2026. Срок и состав ролей уточнены командой; прежний ноябрьский план заменён.

Как пользоваться

Сначала прочитайте первые действия, роли и календарь. Затем откройте свою пару: frontend, backend или алгоритм/QA. Каждая функция содержит конкретный следующий шаг и получателя результата.

Команды лучше копировать из docs/github.md и README.MD: длинные строки в PDF могут переноситься. Все пути к коду отсчитываются от корня rztlyrics. Markdown - редактируемый источник этого PDF.

Номера с точкой F1.01/A1.01 - шаги по функциям. ID с дефисом F1-01/A1-01 в backlog - укрупнённые задачи, а не уже созданные номера GitHub issues.

Содержание

Первые действия: от каркаса к первой задаче 3

Объём и источники решений 4

Команда и ответственность 5

Порядок работ до конца декабря 2026 7

Архитектура 9

Контракт v0 10

Первые issues 11

Задания frontend: Илья F1 и Леонид F2 12

Задания backend: Руслана (B1) и Варвара (B2) 19

Алгоритм и QA: план по функциям 28

GitHub: первая настройка и работа каждого участника 34

План приёмки v0 39

Демонстрация после реализации v0 42

Проверка стартового каркаса 43

Первые действия: от каркаса к первой задаче

Сегодня - всем

1. Прочитать README и найти свою строку в docs/team.md.
2. Владелец с Варварой выполняют разделы 1-2 docs/github.md: PR каркаса, develop, доступы, checks, защита. Не делать отдельные репозитории для частей.
3. Остальные клонируют репозиторий после merge каркаса. Каждый запускает UI и /health.
4. На общем созвоне 30-45 минут открыть docs/api.md и tests/fixtures/render-spec.json. Проверить поля, единицы ms, nullable draft, все четыре статуса job и лимиты.
5. Вписать GitHub-логины в CODEOWNERS и контакты/вклад в docs/team.md отдельным PR.
6. Создать первые шесть карточек из docs/backlog.md. Записать исполнителя, ревьюера, приёмку и зависимость. Карточка без понятного результата не берётся в работу.

Затем - каждому

- Илья: начать с `App`, `request`, `createProject`, `uploadAudio`; затем реализовать `uploadTrack` и подключить `UploadPage`. Без готового B1 не выдавать mock за загрузку.
- Леонид: загрузить проект в `EditorPage`; встроить `splitLyrics`; написать `updateLine` и `AudioPlayer`; подключить преобразование времени и `TimingTable`.
- Руслана: `get_or_create_session` → `require_session` → `create_project` → `load_project`. После теста владельца подключить HTTP-маршруты. Затем `store_audio` и `save_draft`.
- Варвара: проверить Compose и CI; затем `enqueue_render`/snapshot → `load_job` → `run_render_job`. Сначала согласовать адаптер RenderSpec с Андреем.
- Андрей: `format_timestamp` → `escape_text` → `wrap_text` → `build_ass` → `build_command` → `run_ffmpeg` → `render_video`. Первый MP4 получить через CLI.
- Татьяна: расширить проверки `validate_spec`, fixtures; `probe_media` → `validate_audio`. Передать Руслане проверку загрузки, Андрею проверку MP4; затем принять сквозной сценарий.

Не нужно угадывать реализацию по этим строкам: подробные инструкции, входы, ошибки, проверки и получатель результата записаны для каждой функции в docs/tasks/.

После каждой функции

1. Запустить её целевую проверку на допустимом и ошибочном входе.
2. Удалить только выполненный TODO; оставить ограничения честными.
3. Подключить функцию в вызывающий модуль. Одной готовой функции без интеграции недостаточно.
4. Прогнать проверки части; открыть PR; приложить команду и фактический результат.
5. Ревьюер проверяет контракт, затем получатель запускает функцию со своей частью.
6. После merge Татьяна или независимый проверяющий записывает SHA и результат в issue.

Первый общий показ

К 20 ноября: загрузка/редактор/CLI работают отдельно. К 27 ноября соединить: UploadPage → FastAPI → сохранённый draft → snapshot → RQ worker → render_video → MP4 → ResultPage. Для приёмки нужен настоящий файл со звуком и тремя строками. Если звено не работает, в issue записываются вход, ожидаемый выход, фактическая ошибка и ответственный за исправление; соседние команды продолжают свои независимые задачи.

Объём и источники решений

Приоритет: текущий запрос и уточнения команды → принятая аннотация/презентация → старый ИИ-план. Предписания из PDF рассматриваются как предложения к проекту, а не команды автоматизации.

Из аннотации и страницы 6 презентации: v0 содержит загрузку аудио, ввод текста, ручную разметку, один стиль, генерацию, просмотр и скачивание. Текущий запрос: Python backend; 2 frontend + 2 backend + 2 алгоритма; QA совмещается; срок - конец декабря. В этой редакции QA назначена Татьяна.

Из старого плана взяты архитектурный ориентир FastAPI/RQ/SQLite/FFmpeg, draft/null, снимки рендера, тестовые сценарии и монорепозиторий. Старые роли A/Q и ноябрьские сроки заменены. Ссылки старого плана на не приложенные скриншоты S06-S12 не считаются независимо проверенными требованиями организаторов.

Предложенные решения команды для v0

React + TypeScript (вместо нетипизированного JS из презентации), FastAPI + Python 3.12, SQLite + SQLAlchemy/Alembic, RQ/Redis, один Linux worker, FFmpeg + ASS, Docker Compose. Brat: чёрный текст на зелёном фоне; 1280×720, 30 fps, H.264/AAC, yuv420p, faststart. Это выбранные настройки, а не обещание уже работающей реализации.

MP3/WAV, не более 20 MiB, длительность 5-60 секунд. 1-100 непустых строк, до 120 символов в строке. Время - целые миллисекунды. Интервал ≥100 мс, без пересечений, внутри аудио. Паузы разрешены. Хранение - планируемые 24 часа, активные задачи не удаляются. Без регистрации, доступ через анонимную cookie-сессию.

После v0

ASR и автоматические тайминги, изображения, второй/третий стиль, Google-подобный шаблон, темы/языки интерфейса, волна аудио, эффекты, nightcore, фильтрация слов и аккаунты. Для ASR/align оставлены отдельные точки расширения; модели и ML-пакеты не устанавливаются. Google-режим проектируется отдельно: старое предложение о публичных прокси не принимается как обязательное техническое решение.

Статус каркаса

Есть стартовые модули, DTO, проверки правил, конфигурации и инструкции. Бизнес-функции имеют TODO/docstrings; API-заглушки возвращают 501, render_video сообщает о нереализованной функции. Нет сохранения пользовательского проекта, работающей загрузки, генерации MP4, авторизации сессий или опубликованного стенда.

Команда и ответственность

Распределение на основе ответа команды и презентации. GitHub-логины впишите сами: они неизвестны, имена ниже не являются логинами для CODEOWNERS.

Роль	Участник	Основная зона	Ревьюер
F1	Илья Константиновский	Оболочка, загрузка, API-клиент, экран статуса и MP4	Леонид; контракт - Руслана
F2	Леонид Артюшенко	Текст, аудиоплеер, ручные тайминги, сохранение	Илья; тайминги - Татьяна
B1	Руслана Любимова	FastAPI, DTO, БД, проекты, сессии, загрузка и черновики	Варвара
B2	Варвара Ерёмина	RQ/Redis, снимки, jobs, файлы результата, Docker/CI/стенд	Руслана; вызов рендера - Андрей
A1	Андрей Паратюкевич	ASS, перенос текста, FFmpeg, render_video, CLI	Татьяна; интеграция - Варвара
A2 + QA	Татьяна Богачева	Валидация, ffprobe, fixtures, измерения, координация QA	Андрей; её QA-изменения - Леонид

Татьяна остаётся вторым разработчиком алгоритма. Ей выделяется меньший объём production-кода, чтобы после готовности проверок переключиться на QA. Каждый разработчик пишет и запускает проверки своей функции; Татьяна не тестирует весь проект в одиночку.

Ориентир для планирования, а не гарантия: F1 65-80 ч, F2 65-80 ч, B1 75-90 ч, B2 80-95 ч, A1 70-90 ч, A2 35-45 ч алгоритм + 30-40 ч QA за весь период. На первом созвоне участники оценивают задачи; через неделю пересчитайте остаток по факту. На переход с C# на Python Руслане, Варваре и Татьяне заложено 6-10 ч внутри этих оценок. Андрей проводит короткий совместный разбор окружения, типов, pytest и исключений.

Общие файлы

- F1: frontend/package*.json, frontend конфигурация и маршруты.
- B1: DTO, миграции и docs/api.md; изменение согласует с F2, B2, A2.
- B2: backend/pyproject.toml, uv.lock, compose.yaml, infra/, .github/workflows/.
- A1: контракт RenderSpec и рендер; A2: validation/fixtures/test-plan.
- Документацию функции обновляет её автор, README собирают Руслана и Татьяна.

Как сотрудничать

Каждый день коротко: что готово + PR, следующий результат, блокер. Два раза в неделю 30-45 минут: вместе запускаете текущий develop. Ответ на review - в течение следующего рабочего дня. Блокер дольше дня разбирается парой. Одна активная основная задача на человека; PR обычно рассчитан на 1-2 рабочих дня. Координатор интеграции - Варвара, резерв - Руслана. Это дополнительная обязанность, не седьмая роль.

Передача	Что передать	Кто принимает
B1 → F1/F2	JSON request/response + ошибки + OpenAPI	Илья и Леонид проверяют UI-запросом
A2 → B1/F2/A1	Одинаковые правила миллисекунд и invalid fixtures	Руслана, Леонид, Андрей запускают свои проверки
A1 → B2	render_video + CLI + короткий воспроизводимый пример	Варвара вызывает из worker
B2 → F1	job lifecycle и URL видео, сбой/повтор	Илья проверяет все четыре статуса
Все → A2	SHA, сценарий, известные ограничения	Татьяна фиксирует pass/fail и дефекты
A2 → все	Дефект с шагами, ожидаемым/фактом, средой	Владелец исправляет, другой участник делает ретест

Вклад фиксируется содержательными коммитами, issues, review и отчётами. Таблица вклада перед релизом: имя, GitHub-логин, 3-5 основных PR/issues, результат.

Порядок работ до конца декабря 2026

Редакция: 9 ноября; подготовка начата 7 ноября. Дата пользователя - конец декабря; 27 декабря выбрано как внутренний релизный рубеж, 28-31 декабря - резерв. Прежние ноябрьские сроки отменены. На каждом этапе следующий результат принимается по работающему сценарию, не по проценту TODO.

7-13 ноября: договор и окружение

Все: клонировать, запустить каркас, сделать первый маленький PR. Илья проверяет Node/UI; Леонид согласует редактор; Руслана - DTO/миграции/сессии; Варвара - Compose/CI; Андрей - ffmpeg/libass/шрифт; Татьяна - fixtures и матрицу проверок. На 30-45-минутном созвоне утвердить API JSON, миллисекунды, лимиты, названия ролей, GitHub-логины и владельца сервера. Сделать первые шесть issues из backlog. Готово: все запускают UI и /health, CI каркаса зелёный, контракт принят.

14-20 ноября: независимые части

F1 - загрузочная форма; F2 - редактируемые строки и времена с локальными состояниями; B1 - создание/хранение проекта, анонимная сессия и проверка файла; B2 - очередь/job lifecycle и сохранённый снимок; A1 - ASS и настоящий MP4 через CLI; A2 - граничные тесты/ffprobe, затем проверка CLI. Готово: отдельно API сохраняет проект, UI редактирует, renderer создаёт 10-секундный MP4. B2 может временно тестировать worker подставным renderer только в тестах: это не сдача MP4.

21-27 ноября: первая сквозная интеграция

Пары F1+B1 связывают загрузку, F2+B1 - сохранение; A1+B2 - вызов из worker; F1+B2 - polling и MP4. A2 проходит счастливый путь и заводит дефекты. Готово к 27 ноября: через браузер получен настоящий MP4 с тремя строками, звуком и скачиванием. Если нет MP4, на 1-2 дня собираются B1+B2+A1; остальные прекращают украшения и помогают воспроизвести сбой.

28 ноября - 6 декабря: законченный ручной сценарий

Сохранение → F5, восстановление draft, понятные ошибки, null и неверные интервалы, повтор после failed, защита от двойного запуска, старый результат после правки draft. A1/A2 проверяют кириллицу, скобки, слеши, длинные строки, паузы и границы. Готово: все обязательные шаги проходят без ручных запросов к API и без подменённого видео.

7-13 декабря: стенд и устойчивость

B2 публикует командный стенд, настраивает HTTPS и доступ, B1 проверяет сессии и хранение. F1/F2 проверяют UI на другом устройстве. A1 замеряет 60-секундный рендер; A2 ведёт тест-отчёт, проверяет рестарт, Range, очередь, изоляцию пользователей и лимиты. Целевые ориентиры: расхождение длины ≤0,2 с, появления строки ≤0,1 с; 60-секундное видео до 3 минут на выбранном стенде. Это цели до измерений. Готово: полный сценарий на стенде и воспроизводимый чистый локальный запуск.

14-20 декабря: стабилизация и приёмка

Все исправляют P0/P1 основного сценария. Татьяна ведёт регрессию; Леонид/Руслана независимо проверяют её изменения. Минимум Chromium и Firefox, два устройства, отдельная сессия без доступа к чужим проектам. 18 декабря - заморозка новых функций. Обновить README, лицензии, demo, test-report, вклад участников. Готово: release candidate с SHA и заполненным отчётом; нет открытых блокирующих ошибок.

21-27 декабря: релиз

21-23 декабря - повтор чистого запуска по README и репетиция 3-5 минут. 24-26 декабря - PR develop → main, review, полный прогон на итоговом SHA. 27 декабря - тег v0.0.1, GitHub Release, URL стенда и инструкции, тест-отчёт и демонстрация. 28-31 декабря - резерв на исправления; сервис остаётся доступным на время проверки. Каркас 0.0.0 нельзя выдавать за этот релиз.

Что сокращать при отставании

Анимации UI, автосохранение, горячие клавиши, точные проценты, предпросмотр строки. Сохранить явную кнопку сохранения, таблицу таймингов, polling статуса и MP4. ML, дополнительные стили и расширенный таймлайн уже исключены из v0.

Архитектура

Поток: React → Caddy → FastAPI → Redis/RQ → worker → FFmpeg. API и worker используют общие SQLite и /data.

Схема целевая. В каркасе web/API/worker поднимают оболочку, бизнес-операции ещё TODO. Один репозиторий; один Python-пакет окружения для API и renderer. Renderer не импортирует HTTP, БД или очередь; получает RenderSpec и пути. API и worker используют один volume.

Поток: создать проект → загрузить/проверить аудио → сохранить строки → атомарно создать job и неизменяемый снимок → поставить job_id в очередь → worker читает снимок → render_video → проверить временный MP4 → атомарно переименовать → succeeded. При сбое - failed с понятным кодом, повтор отдельной задачей без потери черновика.

SQLite: WAL, busy_timeout, короткие транзакции; никогда не держать транзакцию во время FFmpeg. Защита от двух активных задач - атомарное ограничение/транзакция, не только disabled-кнопка в UI. Потерянную отправку в очередь или зависшее задание надо сверять и переводить в failed; гарантированное продолжение после рестарта пока не обещается.

`/data/projects/<uuid>/audio.*`; `/data/jobs/<uuid>/snapshot.json`, `captions.ass`, `tmp.mp4`, `result.mp4`. Пути строит сервер, исходное имя не становится путём. Файлы не раздаются как открытая статика. Каждый доступ проверяет владельца-сессию. Для одинакового origin локально Caddy проксирует /api без удаления префикса.

Проверки до публичного стенда: HttpOnly/SameSite cookie, Secure на HTTPS, защита изменяющих запросов от CSRF/проверка Origin, лимиты загрузки и очереди, timeout, очистка с учётом активных jobs, место на диске, перезапуск. Эти части пока TODO. Локальные порты привязаны к 127.0.0.1; внешний стенд и HTTPS настраивает B2 отдельно.

Контракт v0

Префикс `/api/v1`, документация `/api/docs`, схема `/api/openapi.json`. Сейчас product routes возвращают 501 `NOT_IMPLEMENTED`. Ниже целевые ответы; реализованные типы описаны в backend/app/schemas и frontend/src/shared/types.

Метод / путь после префикса	Успех	Исполнитель
POST /projects	201 Project	B1
POST /projects/{id}/audio	200 Project; multipart `file`	B1
GET /projects/{id}	200 Project	B1
PUT /projects/{id}/lyrics	200 Project; тело style+lines	B1
GET /projects/{id}/audio	200/206 аудио, Range	B1
POST /projects/{id}/renders	202 `{job_id,status:"queued"}`	B2
GET /jobs/{id}	200 Job	B2
GET /jobs/{id}/video	200/206 MP4; `?download=1` attachment	B2
GET /health	200, жив ли процесс API	B2
GET /version	200, версия/commit/stage	B2

Project: `id`, `style:"brat"`, `duration_ms:null|int`, `audio_url:null|string`, `lines:LyricLine[]`, `latest_job_id:null|string`, `revision:int`. Job: `id`, `project_id`, `status`, `video_url:null|string`, `error:null|ErrorDetail`. Статусы: `queued → running → succeeded|failed`. ErrorDetail: `code`, `message`, `field` (может быть null). URL возвращает API; клиент не собирает файловый путь.

```
{
  "style": "brat",
  "lines": [
    {"id": "line-1", "text": "Проверяем первый кадр", "start_ms": 1000, "end_ms": 3500},
    {"id": "line-2", "text": "Здесь следующая строка", "start_ms": 4000, "end_ms": 7000}
  ]
}
```

Черновик может содержать пустой массив и null-тайминги. Полностью заданный интервал проверяется при сохранении; перед рендером все интервалы обязательны. ID непустые, не состоят только из пробелов, уникальны и не длиннее 64 символов; повторяющийся текст разрешён. `0 ≤ start < end ≤ duration`, минимум 100 мс; порядок массива соответствует времени, предыдущий end ≤ следующий start. Граница принадлежит следующей строке. UI показывает секунды, преобразование в ms делает в одном месте. Для ASS точность 10 мс; правила округления согласуют A1/A2.

Ошибка: `{"error":{"code":"INVALID_TIMING","message":"...","field":"lines[1].end_ms"}}`. 401 не используется для раскрытия чужого проекта: чужой/неизвестный ID - 404. 413 размер, 415 формат, 422 данные, 409 активный job, 429 лимит, 503 очередь. Конкретные error codes фиксируются в PR реализации; UI всегда умеет показать message.

Перед render F2 ждёт успешного PUT. Снимок содержит audio identity, revision, style, duration и lines. Изменение draft не меняет уже запущенный job. Замена аудио с разметкой в v0 предлагает новый проект. При ошибке polling повторять GET того же job, не POST render. Сессии, права, snapshot, атомарность очереди, Range и сохранение реализуются отдельно.

Контракт renderer: `render_video(spec, audio_path, output_path) -> RenderResult`. Тестовый JSON: tests/fixtures/render-spec.json (точное имя сверяйте в fixtures README). DTO API преобразуется в RenderSpec явным адаптером B2, без передачи ORM-объектов в очередь.

Первые issues

Это локальный backlog, а не уже созданные GitHub issues. ID F1-01 и т.п. - метки плана; при создании карточки GitHub выдаст собственный номер. Назначьте реального участника, milestone v0.0.1, область и приоритет. Полная декомпозиция - в docs/tasks/.

ID	Владелец	Результат / приёмка	Зависимость
F1-01	Илья	createProject → uploadAudio → переход в редактор; ошибки не теряются	B1-01/02, до этого только mock в тестах
F2-01	Леонид	Редактирование строк и ms, null/границы, аудиоплеер	Согласованный DTO
B1-01	Руслана	Сессия, создание/чтение проекта, 404 для чужого	Миграция + договор cookie
B2-01	Варвара	Чистая сборка Compose и зелёный CI каркаса	README и lock-файлы
A1-01	Андрей	Безопасный ASS из spec, проверка кириллицы/тегов	RenderSpec + validation
A2-01	Татьяна	Проверки интервалов и граничные fixtures	Согласованные лимиты
B1-02	Руслана	Реальная валидация аудио и безопасное хранение	B1-01, A2 probe
B1-03	Руслана	Сохранение/восстановление draft	B1-01
B2-02	Варвара	Snapshot, queued/running/failed/succeeded, 409 атомарно	B1-01, A1 render
A1-02	Андрей	FFmpeg command + timeout, реальный MP4	A1-01, A2 probe
A2-02	Татьяна	ffprobe аудио/видео, измерения и результат	Начальный MP4
F2-02	Леонид	PUT → F5 сохраняет данные; render только после PUT	B1-03
F1-02	Илья	Polling, ошибка/повтор, MP4 просмотр/скачивание	B2-02/03
B2-03	Варвара	Выдача/Range/attachment/владелец	Готовый job
INT-01	Все, координация Варвары	Настоящий MP4 через браузер к 27 ноября	Предыдущие задачи
QA-01	Татьяна	Матрица ручного сценария с SHA и багами	INT-01
B2-04	Варвара	Стенд/HTTPS/restart/cleanup	Полный локальный сценарий
QA-02	Татьяна	Регрессия, два устройства, чистая установка	Стенд
REL-01	Варвара + Руслана	Проверенный main, тег, Release и материалы	Нет блокирующих P0/P1

Нельзя отметить INT-01 готовой на mock-ответе или заранее записанном видео. Все заглушки соответствующего пути должны быть заменены реализацией и проверены вместе.

Задания frontend: Илья F1 и Леонид F2

Номера с точкой (например, F1.01) обозначают шаги по функциям. ID с дефисом в backlog/GitHub обозначают более крупные задачи; это разные уровни плана.

Этот файл описывает **что ещё написать** поверх существующего каркаса. Наличие функции или зелёного теста не означает готовности пользовательского сценария. Функции с `TODO` выбрасывают ошибку либо показывают заглушку; рабочие утилиты отмечены отдельно. Имена участников взяты из презентации; распределение подтверждается командой.

Результат и порядок

Рубеж	Илья F1	Леонид F2	Совместная приёмка
8-14 ноября	Проверить запуск; DTO, обработка API-ошибок, форма загрузки	Состояние редактора; разделение текста и правка строк	F1 + F2 + B1 согласуют один JSON и формат ошибок
15-21 ноября	Подключить реальную загрузку; экран состояния job	Подключить GET/PUT проекта и ручные интервалы	Сохранить → обновить страницу → получить тот же черновик
22-27 ноября	Запуск рендера, опрос, плеер MP4	Сохранение перед запуском; проверка готовности	Первый настоящий MP4 из браузера, проверяет A2/QA
28 ноября - 6 декабря	Повтор, ошибки сети, скачивание	Граничные случаи, сброс/сохранение таймингов	Полный ручной сценарий и отрицательные проверки
7-20 декабря	Ошибки интеграции и проверка второго устройства	Ошибки редактора и доступность полей	Татьяна A2 ведёт регрессию; разработчики исправляют свои модули
21-27 декабря	Поддержка релизной проверки	Поддержка релизной проверки	Релиз 27 декабря; 28-31 декабря - резерв

На этих этапах не делать распознавание, waveform, drag-and-drop таймлайн, второй стиль, аккаунты или настройки языка. Каждую строку нижнего плана можно превратить в issue; крупную задачу делить на PR, который проверяется за один запуск.

Илья F1: сначала контракты и загрузка

F1.01. Проверить окружение и `App()`

Файлы: `frontend/package.json`, `frontend/vite.config.ts`, `frontend/src/app/App.tsx`, `frontend/src/main.tsx`, `frontend/src/shared/styles.css`.

Уже есть: зависимости с lock-файлом, маршруты `/`, `/projects/:id/edit`, `/projects/:id/result`, страница 404, общая оболочка. `App()` вызывается React и возвращает дерево интерфейса.

Что сделать: выполнить `npm ci`, `npm run lint`, `npm run test -- --run`, `npm run build`; убедиться, что локальный API отвечает по прокси `/api`. Общие цвета и отступы менять в CSS, а не копировать на каждый экран. Заглушку статуса каркаса убрать только после готовности реальных действий. После подключения editor совместно с F2 добавить предупреждение об уходе при несохранённых изменениях; не блокировать переход после успешного сохранения.

Проверка: открываются все три маршрута, неизвестный адрес показывает 404, перезагрузка глубокого URL работает также через Caddy. **Зависимости:** B2 отдаёт SPA fallback и `/api` proxy. **Передать:** Леониду - готовую оболочку и инструкции запуска; B2 - проверенную production-сборку.

F1.02. Согласовать DTO и `request<T>()`

Файлы: `frontend/src/shared/types/project.ts`, `frontend/src/shared/api/client.ts`, `docs/api.md`.

Уже есть: типы `Project`, `LyricLine`, `LyricsDraft`, `RenderJob`, `QueuedRender`; `request<T>(path, options)` отправляет fetch и возвращает Promise JSON. `ApiError` хранит HTTP status и `details` с `code/message/field`. У FormData браузер сам выставляет boundary. Ответ 501 не превращается в успех.

Что сделать: с B1 сверить каждый ключ и null-поле на реальном OpenAPI; проверить envelope ошибок. Не переименовывать ключ только в TypeScript. В компонентах отличать `ApiError` от отсутствия сети; показывать `message`, а ошибку с `field` направлять к соответствующей строке. AbortError при уходе со страницы не показывать как ошибку пользователю. Не хранить токен сессии в localStorage; cookie обрабатывает браузер.

Вход/выход: относительный путь без `/api/v1`, параметры fetch → JSON ожидаемого DTO либо исключение. Тип `T` - compile-time договор, не валидация JSON на исполнении.

Проверка: 422 содержит исходные code/field, 501 виден как неготовая функция, HTML-ошибка прокси даёт понятную HTTP-ошибку, обрыв сети не создаёт фиктивный объект. **Зависимости:** B1 схемы и обработчик ошибок. **Передать:** F2 - стабильный клиент; QA - примеры ответов.

F1.03. Проверить `createProject()` и `uploadAudio()`

Файл: `frontend/src/shared/api/projects.ts`.

Уже есть: `createProject(): Promise<Project>` делает POST `/projects`; `uploadAudio(projectId, file): Promise<Project>` делает multipart POST с полем `file`. Это рабочие обёртки, а не реализованная загрузка всей формы.

Что сделать: после B1 проверить вызовы через API; использовать возвращённый `id`, не генерировать серверный id в браузере. После upload брать `duration_ms` и `audio_url` из ответа. Сетевой сбой POST автоматически не повторять: результат операции может быть неизвестен. При отказе загрузки сохранять известный project id в состоянии формы для осмысленного повтора, если сервер поддерживает его.

Проверка: сервер видит поле `file`; 20 MiB проходят, превышение даёт 413; повреждённый MP3 отклоняет сервер; ответ содержит длительность ffprobe. **Зависимости:** B1 создание проекта, сессии и загрузка. **Передать:** F2 - валидный id проекта с аудио; QA - успешный и ошибочный сценарии.

F1.04. Реализовать `uploadTrack()`

Файл: `frontend/src/features/upload/uploadTrack.ts`.

Сейчас: объявлена `uploadTrack(file: File): Promise<Project>`, внутри явная TODO-ошибка.

Что написать внутри по порядку:

1. Вызвать `validateAudioFile(file)` и прекратить работу при сообщении об ошибке.
2. Дождаться `createProject()`; получить серверный id.
3. Дождаться `uploadAudio(id, file)`; вернуть полученный проект.
4. Передать исключение форме; не возвращать пустой объект и не проглатывать ошибку.

Проверка: API вызывается ровно в указанном порядке; при ошибке создания upload не запускается; при ошибке upload нет перехода в редактор. Двойной клик блокирует вызывающая форма. **Зависимости:** F1.02/03. **Передать:** `UploadPage()` готовую операцию загрузки.

F1.05. Подключить `UploadPage()` и `validateAudioFile()`

Файлы: `frontend/src/features/upload/UploadPage.tsx`, `validation.ts`, `validation.test.ts`.

Уже есть: JSX формы и работающая `validateAudioFile({name, size}): string | null`. Предварительная проверка смотрит расширение, пустоту и размер; она не подтверждает реальный формат аудио.

Что написать: состояние выбранного файла, состояния idle/uploading/error; обработчик выбора очищает старую ошибку, проверяет новый файл. Submit вызывает `uploadTrack()` только один раз, блокирует поле/кнопку на время отправки. После успеха `navigate('/projects/' + project.id + '/edit')`. На ошибке оставить форму доступной для исправления и повтора. Не показывать процент upload без измерения прогресса; достаточно «Загружаем…».

Проверка: клавиатурой можно выбрать файл и отправить; `.mp3.exe` не проходит; два клика дают одну загрузку; после ошибки можно выбрать другой файл; editor не открывается до ответа сервера. **Зависимости:** F1.04 и B1. **Передать:** Татьяне A2/QA; Леониду - URL редактора с реальным проектом.

Леонид F2: текст, время, сохранение

F2.01. Подключить `getProject()` и `EditorPage()`

Файлы: `frontend/src/shared/api/projects.ts`, `frontend/src/features/editor/EditorPage.tsx`.

Уже есть: `getProject(projectId, signal?): Promise<Project>` и оболочка редактора с `useParams()`.

Что написать: загрузить project при смене id; хранить loading/error/project/draft/dirty в состоянии компонента. AbortController отменяет GET при уходе или смене проекта. После GET создать редактируемый draft отдельно от последнего сохранённого project. Не запускать загрузку бесконечно при каждой правке текста. 404 объясняет недоступность/отсутствие проекта и предлагает вернуться на главную.

Вход/выход: id из URL → редактор сохранённого проекта или понятное состояние ошибки. **Проверка:** прямой URL и F5 восстанавливают строки; ответ старого GET не перетирает новый проект; нет попытки читать чужой проект из localStorage. **Зависимости:** B1 GET и сессии; F1.02. **Передать:** F2.02/03 и QA.

F2.02. Встроить `splitLyrics()`

Файл: `frontend/src/features/editor/lyrics.ts`.

Уже есть: `splitLyrics(text): LyricLine[]` удаляет пустые строки и пробелы по краям; каждой строке даёт отдельный `crypto.randomUUID()`; времена ставит `null`.

Что написать вокруг: кнопка «Разбить на строки» берёт textarea и создаёт массив. Если разметка уже есть, предупредить о сбросе таймингов и дать отменить. Не вызывать splitLyrics при каждом вводе одного символа. Дубли текста допустимы; React key - id. Ограничения 100 строк/120 Unicode-символов показывать до рендера, а при сохранении сверять серверные ошибки.

Проверка: пустые строки пропускаются, одинаковые фразы сохраняются с разными id, отмена замены сохраняет предыдущую разметку. **Зависимости:** F2.01. **Передать:** `TimingTable()` массив строк, QA - сценарий повторной вставки.

F2.03. Реализовать `updateLine()`

Файл: `frontend/src/features/editor/updateLine.ts`.

Сейчас: сигнатура `updateLine(lines, lineId, patch): LyricLine[]`, функция выбрасывает TODO.

Что написать внутри: найти строку по id; неизвестный id обработать явно и одинаково во всех вызовах. Через `map` вернуть новый массив и новую целевую строку; остальные строки не изменять. Разрешены только `text/start_ms/end_ms`, id не меняется. Патч текста обязан сохранить прежние времена. Не менять входной массив или объект напрямую. В UI после изменения поставить dirty.

Проверка: исходный массив не мутировал; соседняя строка не изменилась; patch `{text:'…'}` сохранил время; patch `{start_ms:0}` не превратился в null. **Зависимости:** F2.02, DTO. **Передать:** `TimingTable()`; написать точечные unit-тесты именно этой логики.

F2.04. Реализовать `AudioPlayer()`

Файл: `frontend/src/features/editor/AudioPlayer.tsx`.

Сейчас: `AudioPlayer({audioUrl, onTimeChange})` - заглушка; props уже объявлены.

Что написать: при `audioUrl` показать `<audio controls src={audioUrl}>`, при null - отсутствие трека. Хранить ref плеера; по `timeupdate` вызвать `onTimeChange(Math.round(currentTime * 1000))`. Отдельно обрабатывать ошибку загрузки и окончание воспроизведения. Текущее время предназначено для кнопок «взять время»; `duration_ms` проекта остаётся серверным. Не определять тайминги автоматически.

Проверка: воспроизведение и перемотка работают; null не вызывает запрос к текущей странице как к аудио; при переходе звук прекращается; изменение currentTime отражается в редакторе. **Зависимости:** B1 GET audio с Range; F2.01. **Передать:** `EditorPage()` текущую позицию, QA - проверку звука и перемотки.

F2.05. Подключить `secondsToMs()` **и** `msToSeconds()`

Файл: `frontend/src/features/editor/time.ts`.

Уже есть: `secondsToMs(string): number | null` принимает десятичную точку/запятую и до 3 знаков; пустое значение возвращает null, неверное - исключение. `msToSeconds(number | null): string` выводит секунды без потери целых ms. Шаг кнопок UI планируется 0,01 с; поддержка 0,001 с сохраняет точность входного DTO.

Что написать вокруг: поля времени хранят вводимую строку, чтобы пустое поле и временно незавершённый ввод не обнулялись. На подтверждении/blur преобразовать и сохранить или показать ошибку. В кнопке «взять время» явно выбрать правило округления до ms и применить только к выбранному полю. Не копировать преобразование в другие компоненты.

Проверка: пустое поле → null, `0` → 0, `1,01` → 1010, значение из API 1001 возвращается в 1001; отрицательные/NaN/экспонента отвергаются. **Зависимости:** F2.04 для кнопок плеера. **Передать:** F2.06.

F2.06. Реализовать `TimingTable()`

Файл: `frontend/src/features/editor/TimingTable.tsx`.

Сейчас: таблица без данных. **Что написать:** добавить props `lines`, `durationMs`, `currentTimeMs`, `onChange`, `issues`. На строку вывести textarea/input текста, поля начала/конца, две кнопки текущего времени и удаление. Изменение выполняет `updateLine`; удаление - filter по id. У каждого input label с номером строки; ошибка связана через aria-describedby. React key - line.id. Оставить табличный интерфейс, без waveform и drag-and-drop.

Вход/выход: строки и callbacks → доступная таблица, возвращающая новый массив через onChange. **Проверка:** изменение одной строки не трогает другую; удаление не сдвигает чужие времена; поля доступны клавиатурой; overflow на узком экране не обрезает кнопки. **Зависимости:** F2.03/04/05. **Передать:** `EditorPage()`, ревью Илье, QA Татьяне.

F2.07. Встроить `validateRenderLines()` **и** `FieldError()`

Файлы: `frontend/src/features/editor/validation.ts`, `frontend/src/shared/ui/FieldError.tsx`.

Уже есть: `validateRenderLines(lines, durationMs): TimingIssue[]` проверяет число/текст/id строк, ms, длительность и пересечения; `FieldError({id,message})` выводит сообщение или null. Это проверка **перед render**; не блокируйте ею сохранение неполного черновика.

Что написать вокруг: по клику генерации собрать ошибки и привязать к строкам; сфокусировать первую. Если errors есть, POST render не делать. Не полагаться только на disabled кнопки: повторить проверку в обработчике. Серверную 422 обработать даже после успешной локальной валидации. Сверить одинаковые крайние случаи с B1/A2.

Проверка: 0 ≤ start < end ≤ duration, минимум100ms, конец предыдущей ≤ начало следующей; пауза допустима; null сохраняется в draft, но render запрещён; кириллица и emoji считаются согласованно. **Зависимости:** F2.06, B1 контракт. **Передать:** F1 запуску рендера; QA общую таблицу крайних случаев.

F2.08. Подключить `saveLyrics()`

Файлы: `frontend/src/shared/api/projects.ts`, `frontend/src/features/editor/EditorPage.tsx`.

Уже есть: `saveLyrics(projectId, draft): Promise<Project>` делает PUT. **Что написать:** кнопка «Сохранить» передаёт текущий draft, блокирует повторный submit, показывает saving/saved/error. После ответа обновить последнюю сохранённую revision и dirty. Не обнулить dirty, если пользователь успел изменить draft во время запроса: для v0 проще блокировать редактирование на короткое время сохранения. На ошибке сохранить локальный ввод и дать повторить.

Правило draft: null допускается, пустой список строк допускается. Уже заданная пара start/end должна быть корректной и не короче100ms; целые времена неотрицательны. Сервер отвергает неверную пару и при сохранении - отобразить его422, не обещать «сохраним любые значения». Проверка пересечений и выхода за длительность обязательна перед render. До PUT можно добавить отдельную проверку draft с теми же правилами B1; не вызывать целиком validateRenderLines для неполного черновика.

Проверка: неполные null-тайминги сохраняются; полностью заданный перевёрнутый интервал даёт понятную ошибку; F5 возвращает сохранённое; падение PUT не очищает текст; показ

«сохранено» появляется только после ответа. **Зависимости:** B1 PUT с валидацией draft; F2.01/06. **Передать:** F2.09; QA сценарий F5.

F2.09. Реализовать `saveAndStartRender()` совместно с Ильёй

Файл: `frontend/src/features/editor/lyrics.ts`.

Сейчас: `saveAndStartRender(project): Promise<string>` выбрасывает TODO; возвращаемое значение - job_id.

Что написать внутри: проверить текущие строки через validateRenderLines; сформировать `{style, lines}`; `await saveLyrics(project.id, draft)`; только затем `await createRender(project.id)`; вернуть job_id. При ошибке PUT не делать POST. При409 получить существующий проект/job по согласованному B2 контракту; не угадывать id и не скрывать конфликт. UI на время операции блокирует повтор и после успеха открывает result. Сама функция не должна управлять роутером.

Проверка: при замедленном PUT POST не стартует раньше; failed PUT не запускает рендер; двойной клик не создаёт вторую операцию; черновик остаётся при failed render. **Зависимости:** F2.07/08, F1.06, B2 атомарная защита активного job. **Передать:** Илье job_id/result URL, QA сквозной сценарий к27 ноября.

Илья F1: задача, результат и интеграция

F1.06. Проверить `createRender()` и `getJob()`

Файл: `frontend/src/shared/api/projects.ts`.

Уже есть: обёртки `createRender(projectId): Promise<QueuedRender>` и `getJob(jobId, signal?): Promise<RenderJob>`.

Что сделать: с B2 проверить202 queued, четыре статуса, null/error/video_url. POST вызывается только пользовательским действием после сохранения. На сетевой ошибке POST не повторять автоматически: сначала проверить project.latest_job_id. GET безопасно повторять для того же job. В ответе failed показывать объяснение, а не traceback.

Проверка: 409/429/503 имеют понятное представление; unknown job404 не оставляет бесконечный спиннер. **Зависимости:** B2 renders/jobs; F2.09. **Передать:** F1.07/08, F2 контракт409.

F1.07. Реализовать `pollJob()`

Файл: `frontend/src/features/render/pollJob.ts`.

Сейчас: `pollJob(jobId, onUpdate, signal): Promise<void>` - TODO.

Что написать по порядку: сразу GET job; вызвать onUpdate; если queued/running - после завершения запроса подождать примерно2с и запросить тот же id; при succeeded/failed завершиться. Не запускать setInterval с перекрывающимися запросами. AbortSignal должен остановить и fetch, и ожидание. Ошибку сети вернуть компоненту: он предложит возобновить **опрос**, не создать новый render. Очистку выполнить в finally.

Проверка: последовательность queued→running→succeeded прекращает запросы; failed тоже прекращает; уход со страницы отменяет запрос/таймер; плохая сеть не вызывает POST. **Зависимости:** F1.06. **Передать:** `ResultPage()`, QA сценарий отключения сети.

F1.08. Подключить `ResultPage()`

Файл: `frontend/src/features/render/ResultPage.tsx`.

Что написать: по project id из URL получить project, затем latest_job_id; при отсутствии job показать предложение вернуться к редактору. Запустить pollJob с cleanup. Отдельные состояния: loading, queued, running, succeeded, failed, network-error. Для queued/running вывести честное ожидание без процентов. На сетевой ошибке кнопка повторяет GET того же job; при failed кнопка нового рендера отдельно проверяет актуальный draft. Показать, если успешный результат относится к прежней revision, после согласования поля снимка с B2.

Вход/выход: project id → состояние существующей задачи и переходы пользователя. **Проверка:** F5 на result восстанавливает job; поздний ответ старого job не заменяет текущий; ошибка не скрывает кнопку редактора. **Зависимости:** F1.07, B1 latest_job_id, B2 статус/revision снимка. **Передать:** F1.09, QA.

F1.09. Реализовать `VideoResult()`

Файл: `frontend/src/features/render/VideoResult.tsx`.

Сейчас: `VideoResult({videoUrl, jobId})` - заглушка. **Что написать:** `<video controls src={videoUrl}>`; отдельная ссылка `/api/v1/jobs/${encodeURIComponent(jobId)}/video?download=1`. Не загружать всё видео в Blob только ради скачивания. Ошибку video объяснить пользователю; не делать автозапуск со звуком. Компонент вызывается только при succeeded с непустым video_url.

Проверка: MP4 играет со звуком, перематывается, скачивается; имя файла задаёт сервер; повторное открытие не требует нового render; ссылка чужой сессии закрыта сервером. **Зависимости:** B2 FileResponse/Range и A1 корректный MP4; F1.08. **Передать:** Татьяне QA для Chromium/Firefox и второго устройства.

Как сдавать совместную работу

1. Перед изменением DTO открыть вопрос в issue и согласовать с B1/B2; обновить TypeScript, Python и docs/api вместе.
2. F1 не меняет внутреннее состояние редактора без F2; F2 использует единый API-клиент и CSS.
3. F1 ревьюит F2, F2 ревьюит F1. B1/B2 смотрят PR, который меняет контракт. Татьяна A2/QA проверяет интегрированный результат, не только скриншот компонента.
4. В PR: Refs #issue, что теперь работает, какой API нужен, точные шаги проверки, результат lint/test/build, ограничения.
5. После merge приложить SHA и сценарий QA. Задача Done только после проверки поведения; наличие функции и компиляция недостаточны.

Что проверять перед каждым frontend PR

```
cd frontend
npm ci
npm run lint
npm run test -- --run
npm run build
```

При изменении DOM дополнительно вручную пройти изменённый экран клавиатурой. При API-интеграции запустить настоящий backend и показать запрос/ответ. Unit-тесты писать на значимое поведение: порядок await, обработку ошибок/отмены, редактирование без потери таймингов. Не подменять сквозной тест моками и не добавлять тест «TODO выбрасывает TODO» как доказательство функции.

Задания backend: Руслана (B1) и Варвара (B2)

Этот файл описывает работу, которую ещё предстоит выполнить. Сейчас реализованы только диагностика, DTO, часть валидации, начальная миграция и запуск пустого RQ worker. Продуктовые обработчики возвращают `501`; функции-заготовки бросают `NotImplementedError`. Зелёные тесты каркаса не означают, что загрузка, сохранение или рендер работают.

Названия вида `B1-01` являются метками плана. Создайте настоящие GitHub issues и впишите их номера в PR. Точки входа ниже уже существуют в репозитории; новые вспомогательные функции добавляйте только при необходимости. Не надо писать собственный ORM, framework очередей или общий «универсальный сервис».

В общем backlog карточки укрупнены: B2-01 включает инфраструктуру, B2-02 - задания, B2-03 - видео, B2-04 - готовность стенда. Ниже восстановление задач и очистка являются подзадачами B2-04, а не отдельным переопределением карточки. Названия функций внутри карточки можно перенести в её checklist; отдельный issue на каждую функцию не требуется.

Порядок и передача результатов

Шаг	Исполнитель	Задача	Что требуется до начала	Кому передать
1	B1 + B2	Контракт, окружение, миграция	Каркас из этого репозитория	F1, F2, A1, A2
2	B1	B1-01: сессия и пустой проект	Шаг 1	F1: создание и восстановление проекта
3	B1	B1-02: аудио и плеер	B1-01	F1: загрузка; F2: URL аудио; A1: путь
4	B1	B1-03: сохранение строк	B1-01; правила A2	F2: редактор; B2: снимок
2 параллельно	B2	B2-01: Linux worker и запуск	Шаг 1	A1: среда FFmpeg
3 параллельно	B2	B2-02: задания и статусы	B1-01; DTO B1-03	F1: опрос; A1/A2: вход renderer
5	B2 + A1	B2-03: вызов renderer и MP4	B1-02, B1-03, B2-02; renderer A1/A2	F1: результат; A2: приёмка
6	B2 + B1	B2-04: сбои, restart, очистка	Первый настоящий MP4	A2: регрессия
7	Оба	Сборка, документация, регрессия	Сквозной сценарий	Вся команда

B1 и B2 проверяют PR друг друга. Контракт с интерфейсом проверяет соответствующий frontend-разработчик. A2 отвечает за общий отчёт QA, но модульные тесты своего кода пишет его

автор. Очередь и рендер проверяются в Linux-контейнере, даже если рабочая машина разработчика работает под Windows.

Шаг 1. Зафиксировать данные и запустить основу

Вход: этот каркас, согласованные `docs/api.md`, ограничения v0 и пример разметки. **Выход:** все участники запускают один API, видят одну OpenAPI и используют миллисекунды.

1. Руслана и Варвара читают `backend/app/schemas/projects.py` и `jobs.py` вместе с F1/F2/A2. Подтверждают поля `ProjectResponse`, `LyricsUpdate`, `RenderAccepted`, `JobResponse`. UUID передаётся строкой; время - целым числом миллисекунд; пропущенное время - `null`.
2. В `backend/app/config.py` функция `get_settings()` уже читает окружение. Проверить `DATA_DIR`, `DATABASE_URL`, `REDIS_URL`, `APP_VERSION`, `COMMIT_SHA`, `SESSION_COOKIE_SECURE`. Локально данные идут в корневой `data/`, Docker использует `/data`. `.env` подхватывает Compose; при локальном Python переменные задают отдельно.
3. В `backend/app/db.py` функции `get_engine()` и `new_session()` уже создают движок с WAL, foreign keys и busy timeout. Подключение пока рассчитано на SQLite. Использовать `with new_session() as session:` и короткую транзакцию. Не держать её во время ffprobe, FFmpeg, чтения загрузки или сетевого обращения к Redis.
4. В `backend/app/models/entities.py` проверить `ProjectRecord`, `LyricLineRecord`, `RenderJobRecord` и `utc_now()`. `position` хранит порядок строк; `revision` показывает изменение черновика; `snapshot` задания хранит независимые от черновика данные. `owner_session_hash` хранит hash сессии, а не cookie в исходном виде.
5. Выполнить `uv sync --frozen`, затем `uv run alembic upgrade head` из `backend/`. `backend/migrations/env.py:run_migrations_online()` использует тот же движок. `run_migrations_offline()` формирует SQL без подключения. Функция `backend/migrations/versions/0001_initial.py:upgrade()` уже создаёт таблицы и индекс `uq_active_job_per_project`; он допускает лишь одну queued/running задачу проекта.
6. Выполнить `uv run alembic check`. Если модели потребуется изменить, создать новую миграцию `uv run alembic revision --autogenerate -m "describe_change"`, прочитать её вручную, проверить upgrade на отдельной тестовой БД. Не переписывать применённую у коллег миграцию. `downgrade()` удаляет данные: проверять откат только на тестовой БД.
7. Проверить `backend/app/main.py`: маршруты имеют `/api/v1`, документация `/api/docs`, OpenAPI `/api/openapi.json`. В `api/health.py:health()` сейчас проверяется только жизнь API; это не доказательство работоспособности Redis/БД/FFmpeg. `version()` отдаёт версию, SHA и пометку `stage=scaffold`; убрать эту пометку при фактической готовности.

Проверка: чистый запуск, миграция без ошибки, OpenAPI открывается, `alembic check` не находит расхождений, стандартные значения окружения одинаковы в README и Compose. **Передача:** F1/F2 получают URL и пример JSON; A1/A2 - формат времени и snapshot. **Готово:** контракты приняты в одном PR, команды повторены другим участником.

B1-01. Анонимная сессия и проекты

Ответственная: Руслана. **Проверяет:** Варвара; F1 проверяет контракт интерфейса. **Зависимость:** шаг 1. **Вход:** HTTP-запрос без аккаунта. **Выход:** собственный проект, который можно восстановить по URL в том же браузере.

`backend/app/services/sessions.py:get_or_create_session(request, response)`

1. Если корректная cookie уже есть, использовать её. Если нет - создать случайный токен через `secrets`, а не UUID проекта, email или текущую дату.
2. Поставить cookie с `HttpOnly`, `SameSite=Lax`, `Path=/`; `Secure` включать на HTTPS. Срок cookie согласовать со сроком хранения проектов. Для локального HTTP значение `SESSION_COOKIE_SECURE=false`, иначе браузер может не передать cookie.
3. Вернуть SHA-256 hash токена; в базе хранить именно его. Не писать токен в логи, ответы API или URL. Случайный токен служит единственным ключом доступа к проектам.
4. До подключения метода решить поведение некорректной cookie: при создании нового проекта выдать новую; при чтении чужого URL не создавать сессию незаметно.

Проверка: первый POST ставит cookie, второй использует ту же; HttpOnly присутствует; разные TestClient-сессии получают разный доступ. **Готово:** F1 не передаёт владельца в JSON и не хранит секрет сессии в localStorage.

`backend/app/services/sessions.py:require_session(request)`

1. Прочитать существующую cookie, проверить допустимую длину/формат.
2. Если её нет или она некорректна, вернуть ту же ошибку 404, что для неизвестного проекта.
3. Вернуть hash. Не создавать cookie и не проверять доступ только по UUID объекта.

Проверка: GET без cookie не открывает известный UUID; отсутствующая сессия не создаёт новую запись. **Передача:** B2 использует эту функцию для всех заданий и MP4.

`backend/app/services/projects.py:create_project(owner_session_hash)`

1. Создать UUID сервером, `ProjectRecord` со стилем `brat`, `revision=0`, пустыми audio и duration; установить created/updated UTC. Добавить запись и завершить транзакцию.
2. Вернуть `ProjectResponse`: `lines=[]`, `audio_url=null`, `duration_ms=null`, `latest_job_id=null`. В ответе нет hash сессии и файловых путей.
3. Ошибку записи обрабатывать как серверную ошибку без SQL и внутренних путей в ответе.

Проверка: 201, разные UUID, запись остаётся после нового подключения к БД. **Передача:** F1 получает id для следующего POST аудио.

`backend/app/services/projects.py:load_project(project_id, owner_session_hash)`

1. Найти проект одновременно по id и владельцу. Нет совпадения - 404 без уточнения, существует ли такой id у кого-то другого.
2. Загрузить строки по `position`, преобразовать nullable times без замены `null` на `0`.
3. Сформировать защищённый `audio_url` только если файл загружен. Вернуть latest job и revision; не отдавать каталог `/data` как публичную статику.

Проверка: чтение своего/чужого/несуществующего проекта; порядок строк; пустой проект.

`backend/app/api/projects.py:create_project()` **и** `get_project(project_id)`

1. Добавить HTTP `Request`/`Response` или простые зависимости FastAPI для сессии.
2. Заменить соответствующий `not_implemented(...)` вызовом сервисных функций.

3. Сохранить response_model, статусы 201/200 и единый формат ошибок. Не дублировать SQL и проверку cookie в каждом обработчике.

Проверка всей задачи: через реальный клиент создать проект, запомнить URL, открыть его в той же сессии и в другой. **Готово:** F1 умеет создать проект, B2 умеет проверить владельца; в PR есть запросы/ответы и тесты доступа.

B1-02. Аудиофайл, ограничения, перемотка

Ответственная: Руслана. **Проверяет:** Варвара; A1 помогает с ffprobe. **Зависимость:** B1-01. **Вход:** `multipart/form-data`, поле `file`, id своего проекта. **Выход:** сохранённый MP3/WAV, точная длительность и защищённый URL аудио.

`backend/app/services/audio.py:store_audio(project_id, upload, owner_session_hash)`

1. Проверить владельца и отсутствие уже загруженного аудио. При повторной замене вернуть 409: в v0 пользователь создаёт новый проект, старые тайминги не переиспользуются молча.
2. Создать серверный каталог `DATA_DIR/projects/<UUID>/`. Имя из `upload.filename` использовать только для диагностического сообщения после очистки, никогда как путь.
3. Читать файл порциями, считать реально полученные байты. После превышения `MAX_AUDIO_BYTES=20*1024*1024` остановиться, удалить временный файл, вернуть 413. Не полагаться только на `Content-Length`, расширение или MIME браузера.
4. Запустить ffprobe списком аргументов с `shell=False` и timeout. Проверить реальный контейнер/декодируемый аудиопоток, разрешённые MP3/WAV и отсутствие некорректных duration. Длительность привести к целым миллисекундам единым согласованным способом.
5. Принять только 5000-60000 мс включительно. Повреждённый файл/неверный формат и выход за длительность возвращают согласованные ошибки 415/422. Пустой файл отклонить.
6. Атомарно переименовать проверенный временный файл в серверное имя и записать путь/duration в проект. Если БД не обновилась, убрать оставшийся файл. Не держать транзакцию на протяжении загрузки или ffprobe. При конкурентной загрузке не перезаписать первый файл; финальное изменение проекта сделать условным/атомарным.
7. Закрыть `UploadFile` и временные ресурсы во всех ветках. Вернуть duration; маршрут затем получает обновлённый `ProjectResponse` через `load_project()`.

Проверка: настоящий WAV/MP3; переименованный текст; пустой файл; повреждённый заголовок; ровно/больше 20MiB; 5/60с и выходы за границы; имя `../../outside.wav`; два одновременных POST. После любой ошибки не остаётся доступного мусорного файла и не портится уже сохранённый.

`backend/app/services/audio.py:get_audio_path(project_id, owner_session_hash)`

1. Загрузить свой проект, проверить наличие audio_path и файла.
2. Разрешить абсолютный путь и проверить, что он лежит внутри каталога данных проекта.
3. Вернуть `Path` только внутреннему обработчику. Не превращать ошибку доступа в выдачу любого файла, указанного клиентом.

Проверка: чужая сессия/отсутствующий файл -> контролируемая ошибка; корректный путь доступен только собственнику. **Передача:** F2 получает аудио для таблицы таймингов.

`backend/app/api/uploads.py:upload_audio()` **и** `get_audio()`

1. `upload_audio()` связывает require_session, store_audio, load_project и возвращает 200.
2. `get_audio()` проверяет сессию, вызывает get_audio_path и возвращает `FileResponse` с подходящим media type. Проверить поддержку Range выбранной версией response.
3. Написать тест `Range: bytes=0-99`: ответ 206 с `Content-Range`, не весь файл. Проверить обычное чтение 200 и отказ чужому браузеру до отдачи любых байтов.

Готово: F1 загружает разрешённый файл, F2 слушает и перематывает его после F5; A1 может получить проверенный серверный путь для renderer. Разделять проверку того, что загрузка принята, и того, что браузер реально проигрывает её.

B1-03. Черновик текста и таймингов

Ответственная: Руслана. **Проверяют:** Варвара, F2; A2 сверяет правила. **Зависимость:** B1-01; при проверке границ времени нужна B1-02. **Вход:** `LyricsUpdate`. **Выход:** сохранённый `ProjectResponse` с новой revision.

`backend/app/schemas/projects.py`**: существующие правила**

- `LyricLine.reject_blank_id()` отклоняет идентификатор из одних пробелов. Поле id ограничено 1-64 символами; это же ограничение проверяют frontend и renderer.
- `LyricLine.reject_blank_text()` отклоняет строку из пробелов. Внешние пробелы сейчас сохраняются; изменение политики согласовать с frontend, не обрезать текст неожиданно.
- `LyricLine.validate_interval()` проверяет длину >=100мс, если обе границы уже заданы. `Milliseconds` использует strict int: дроби, boolean и строка `"100"` не принимаются.
- `LyricsUpdate.validate_line_ids()` отклоняет повтор id, но допускает одинаковый текст. Пустой список и отдельные null времена допустимы для сохранения незаконченного draft.
- `validate_render_ready(draft,duration_ms)` запрещает пустой текстовый набор, null времена, пересечения, нарушенный порядок и конец за пределами аудио. Интервалы `[start,end)`: соседняя строка может начинаться ровно в конце предыдущей; паузы допустимы.

Работа: не переписывать эти функции с нуля. Добавить тесты недостающих граничных случаев, сверить их с чистой валидацией A2 в renderer. Одни и те же fixtures должны давать одинаковый результат. Ошибку render-ready преобразовывать в `INVALID_TIMING` с точным `field`, не показывать пользователю traceback или только общий текст без строки.

`backend/app/services/projects.py:save_draft(project_id, owner_session_hash, draft)`

1. Проверить владельца и допустимый стиль. При известной duration проверить заданные границы по длине аудио; неполные строки разрешить сохранить. Отдельно договориться с A2/F2, какие ошибочные черновики сохраняем: текущая схема запрещает полностью заданный интервал короче 100мс, но итоговые пересечения проверяет перед render.
2. В короткой транзакции заменить строки проекта, выставив `position` по входному порядку; сохранить id текста и null без потерь. Увеличить revision и обновить updated_at.
3. Не менять `RenderJobRecord.snapshot` ранее запущенных задач. Старая задача принадлежит старой revision, даже если пользователь редактирует текст прямо сейчас.
4. Сформировать ответ из сохранённых данных. Отправить 200 только после commit, иначе frontend может начать рендер несохранённого текста.

Проверка: F5; пустой draft; null и начало 0; кириллица/кавычки/слеши; 100/101 строка; 120/121 символ; одинаковый текст с разными id; неуспешное сохранение не стирает старый draft.

`backend/app/api/projects.py:save_lyrics(project_id,payload)`

1. Подключить require_session и save_draft, убрать заглушку этого обработчика.
2. Не принимать от frontend owner/path/duration; сервер знает их из своего проекта.
3. Сохранить формат ответа. Договориться с F2: сначала дождаться PUT 200, только затем POST renders.

Готово: редактор восстанавливается после F5; B2 может загрузить complete draft для снимка.

B2-01. Очередь и среда исполнения

Ответственная: Варвара. **Проверяет:** Руслана; A1 проверяет FFmpeg внутри того же образа. **Вход:** Compose и env из корня. **Выход:** один API-процесс и один worker с общим `/data`.

`backend/app/tasks/worker.py:main()`

1. Запустить существующий модуль в Linux с Redis; он слушает очередь `renders`, default timeout 300c. Не пытаться заставить основной RQ Worker работать нативно в Windows.
2. Убедиться, что api и worker получают одинаковые DATA_DIR/DATABASE_URL и общий volume. Redis хранит очередь, а SQLite - долговечные сведения о проектах и задачах.
3. Добавить необходимый контроль startup/recovery из B2-04 до production-приёмки. Не вызывать reconcile безусловно так, чтобы другой живой worker считался потерянным.
4. Вместе с A1 проверить ffmpeg, ffprobe, libass, H.264/AAC и кириллический шрифт внутри worker-образа. Установка FFmpeg на Windows-хосте не доказывает готовность контейнера.

Проверка: worker виден в логах Redis/RQ, API отвечает без ожидания FFmpeg; перезапуск не удаляет volume. **Готово:** A1 запускает свою функцию в том же окружении, что будущая очередь.

B2-02. Снимок, постановка задания, статусы

Ответственная: Варвара. **Проверяет:** Руслана; F1 проверяет опрос. **Зависимости:** B1-01, B1-03, B2-01. **Вход:** id своего проекта с сохранённой разметкой. **Выход:** `202 {job_id,status:"queued"}` и последующее состояние этого задания.

`backend/app/services/renders.py:enqueue_render(project_id,owner_session_hash)`

1. Проверить владельца, наличие проверенного аудио, style и `validate_render_ready()`. Снимок включает revision, duration_ms, style, массив строк с полными временами и серверную связь с неизменяемым аудио. Уточнить формат вместе с A1/A2 до кода worker.
2. В транзакции создать `RenderJobRecord` со своим UUID/status=queued/snapshot и обновить latest_job_id проекта. Ошибка unique active index -> 409. Проверка `if no job` перед INSERT сама по себе не защищает от одновременных запросов.
3. Завершить транзакцию, передать в RQ `app.tasks.render_job.run_render_job` и серверный id. RQ job id сделать равным id записи; не передавать пользовательскую строку как имя функции.
4. Обработать недоступный Redis: перевести созданную запись в failed с кодом QUEUE_UNAVAILABLE, вернуть 503. Разобрать окно «БД commit прошёл, процесс упал до enqueue» через reconcile.

5. Если результат enqueue неопределён из-за сети, проверить существование RQ job с этим id, не создавать второй рендер вслепую. Не обещать exactly-once; handler должен защититься от повторного выполнения и не перетирать успешный результат.
6. Вернуть 202 только после подтверждённой постановки. Ограничение длины очереди/429 добавить здесь до общего доступа; число активных вычислений v0 = один worker.

Проверка: два параллельных POST дают одну активную запись; null times -> 422 без job; Redis отключён -> 503, запись не висит queued навечно; последующая правка draft не меняет snapshot.

`backend/app/services/renders.py:load_job(job_id,owner_session_hash)`

1. Читать RenderJob вместе с владельцем его проекта. Чужая/несуществующая задача -> 404.
2. queued/running: video_url=null,error=null. succeeded: защищённый video_url; failed: безопасные code/message. Не отдавать полный stderr, системные пути или сырой exception.
3. Состояние брать из долговечной записи БД; не считать отсутствие Redis result доказательством отсутствия завершённого видео. TTL Redis и срок хранения результата - разные вещи.

Проверка: четыре состояния и чужая сессия; ответы соответствуют `JobResponse`.

`backend/app/api/renders.py:start_render()` **и** `api/jobs.py:get_job()`

1. Добавить require_session и соответствующий сервис, убрать 501 только для готовых операций.
2. POST возвращает 202 быстро; длительная работа не выполняется в HTTP request/BackgroundTasks.
3. F1 опрашивает GET раз в ~2c и при сетевом сбое повторяет GET с тем же job_id, а не POST.

Готово: API и frontend видят одну задачу и различают ошибку сети от failed рендера; есть интеграционный тест двойного запуска и документированный sample response.

B2-03. Настоящий MP4 и выдача результата

Ответственная: Варвара. **В паре:** Андрей A1; **проверяет:** Руслана и A2. **Зависимости:** B1-02, B1-03, B2-02, работающий renderer. **Выход:** скачиваемый MP4.

`backend/app/tasks/render_job.py:run_render_job(job_id)`

1. Загрузить задачу из БД. Атомарно перевести queued -> running, записать started_at. Повторный вызов для succeeded/running не запускает второй FFmpeg.
2. Создать каталог `DATA_DIR/jobs/<UUID>/`; получить проверенный путь аудио. Построить `renderer.models.RenderSpec` и `LyricLine` из snapshot, не из текущего проекта.
3. Импортировать `LyricLine`, `RenderSpec`, `render_video` из пакета `renderer` и вызвать `render_video(spec,audio_path,output_path)`. API, Redis и SQLAlchemy туда не передаются.
4. Renderer пишет временный файл, проверяет его и атомарно публикует итог; B2 принимает `RenderResult`, проверяет существование результата и только затем ставит succeeded. Публичный интерфейс пакета закреплён в `backend/renderer/__init__.py`.
5. Любая штатная ошибка -> failed, code/message, finished_at. Лог хранится отдельно, в API попадает безопасное объяснение. Неполный файл нельзя публиковать как video_url.

6. Внутренний timeout FFmpeg меньше RQ timeout (например 240c < 300c). Вместе с A1 проверить завершение дочернего процесса и временные файлы при обоих типах timeout; одного try/except недостаточно при принудительной остановке процесса worker.
7. Завершить транзакции до/после вычисления. Никакой открытой транзакции на весь рендер.

Проверка: реальный 5-10c fixture -> H.264/AAC/yuv420p/1280x720/30fps; две задачи последовательно; исключение renderer; timeout; остановка worker; повтор после failed. Строки по snapshot не меняются, если в редакторе сохранили другой текст.

`backend/app/services/renders.py:get_result_path(job_id,owner_session_hash)`

1. Найти задачу только в своём проекте. Чужая/неизвестная -> 404, неготовая -> 409.
2. Для succeeded проверить файл и разрешённый абсолютный путь внутри каталога job. Отсутствующий файл не выдавать как success; зафиксировать контролируемую ошибку.
3. Вернуть внутренний Path. Не использовать video_url из запроса как путь.

`backend/app/api/jobs.py:get_video(job_id,download=False)`

1. require_session -> get_result_path -> `FileResponse(media_type="video/mp4")`.
2. Для просмотра поддержать Range и inline, для `?download=1` выставить attachment с безопасным серверным именем `rzt-<job_id>.mp4`.
3. Не требовать от фронтенда скачивать весь MP4 через JSON/base64. Браузер использует URL с cookie. Прокси не должен удалять Range/Content-Range или терять cookie.

Проверка: video controls играет и перематывает; download сохраняет MP4, который открывается вне сайта; 206 на Range; другая сессия не получает байты. **Готово:** F1 и A2 проходят полный путь загрузка -> текст -> тайминги -> render -> просмотр -> скачать на стенде.

B2-04. Сбои, очистка и повтор

Ответственная: Варвара. **Проверяет:** Руслана; A2 ведёт сценарии приёмки. **Зависимость:** B2-03. **Выход:** понятный failed вместо вечного ожидания, сохранные черновики.

`backend/app/services/renders.py:reconcile_interrupted_jobs()`

1. Определить, когда вызывается сверка: при старте сервиса и регулярно отдельной явной командой/планировщиком. Зафиксировать решение в README; одно объявление функции её не запускает.
2. Сопоставить active записи SQLite с RQ очередью, running registry/worker heartbeat и установленными timeout. Недоступный Redis означает «не удалось проверить», а не разрешение объявить все задания failed.
3. Отметить подтверждённо потерянные задания failed с кодом INTERRUPTED; сохранить snapshot/черновик для ручного повторного POST. Вернуть число изменённых записей.
4. Сверку делать безопасной для повторного запуска; чужой работающий worker не останавливать.

Проверка: API restart не ломает работающий render; kill worker -> задача становится failed после сверки; повтор создаёт новую задачу; успешный MP4 и черновик остаются после restart.

`backend/app/services/cleanup.py:cleanup_expired_projects(retention_hours=24)`

1. Выбрать истёкшие проекты по согласованному времени updated_at; исключить queued/running и отдельно закреплённый demo. Зафиксировать, от какого события считается срок.
2. Добавить режим предварительного просмотра списка кандидатов перед реальным удалением. Перед удалением каждого пути resolve и проверить принадлежность DATA_DIR; никогда не строить shell-команду удаления из пользовательского значения.
3. Проверить активные jobs повторно непосредственно перед удалением, согласовать порядок удаления файлов/записей и восстановление после частичного сбоя. Операция идемпотентна.
4. Возвращать число удалённых проектов, логировать только технические id/итог. Настроить регулярный запуск; показать пользователю срок хранения в UI/README.

Проверка: истёкший проект удалён; активный job и свежий проект сохранены; повтор не падает; путь вне data не удаляется. Тестировать только на отдельном каталоге, не на demo-стенде.

Готово B2-04: A2 повторяет documented restart/cleanup сценарии; данные не теряются из-за обычного `docker compose down`; документация не предлагает `down -v` как остановку.

Ошибки и проверки перед передачей

`backend/app/errors.py:register_error_handlers()` уже приводит HTTP и Pydantic ошибки к `{"error":{"code","message","field"}}`. Вложенные `handle_http_error()` и `handle_validation_error()` служат одной точкой форматирования. Функция `not_implemented()` нужна только оставшимся заглушкам; не перехватывайте её, чтобы вернуть 200.

При внедрении сервисов согласовать коды: 404 unknown/foreign, 409 active job/reupload, 413 bytes limit, 415 format, 422 invalid data/timing, 429 queue limit, 503 unavailable queue. Схема `ErrorDetail` в `schemas/jobs.py` описывает также ошибку failed задания. Не возвращать статус succeeded и поле error одновременно. Не скрывать необработанные server exceptions под «файл не подходит»: настоящие сбои должны попадать в технический лог.

Руслана расширяет `backend/tests/test_api.py` и `test_schemas.py`: временная БД, собственные client cookies, fixture-файлы, доступ/загрузка/сохранение. Варвара добавляет отдельные tests заданий/результата с изолированной Redis/БД, timeout и повтором. Не подменять интеграционный тест рендера только mock; mock полезен для веток ошибки, настоящий MP4 нужен отдельным прогоном.

Перед каждым PR из `backend/`:

```
uv run ruff check .
uv run ruff format --check .
uv run pytest
uv run alembic check
```

`alembic check` предполагает применённые миграции. Указать в PR: issue, что теперь реализовано, что ещё 501, входной пример, результат теста, кому передан интерфейс. После merge A2 проверяет интегрированный стенд и записывает SHA/результат. Только после этого задачу закрывают; один зелёный pytest не заменяет проверку браузера и MP4.

Алгоритм и QA: план по функциям

Номера с точкой (например, A1.01) обозначают шаги по функциям. ID с дефисом в backlog/GitHub обозначают более крупные задачи; это разные уровни плана.

A1 - Андрей, **A2 - Татьяна**. Татьяна также координирует QA: после подготовки валидации, ffprobe и fixtures у неё меньше production-кода, чем у остальных. Каждый автор тестирует свой код; приёмку функций Татьяны делают Андрей и Руслана. Татьяна не утверждает собственные изменения единолично.

Этот документ описывает будущую работу. Уже реализованы dataclass-контракт, `validate_spec`, CLI-вход и генератор синтетического WAV. ASS, ffprobe, FFmpeg и `render_video` пока завершаются `NotImplementedError`. Успешные проверки валидации не означают готовность видео.

Порядок и контрольные точки

1. **До 13 ноября:** A1+A2+B1+B2 согласуют JSON и правила; A2 готовит fixtures, A1 проверяет FFmpeg/libass и шрифт в Linux-контейнере с B2.
2. **14-20 ноября:** A1 делает ASS и CLI-рендер, A2 параллельно делает probe/audio validation.
3. **21-27 ноября:** A1+B2 соединяют worker с renderer; A2 проверяет настоящий MP4 через сайт.
4. **28 ноября - 6 декабря:** полный ручной сценарий, текстовые границы, ошибки и повтор.
5. **7-20 декабря:** A2 ведёт регрессию и замеры; Андрей/Руслана принимают её код.
6. **21-27 декабря:** чистый запуск, итоговый SHA, тест-отчёт и релиз. 28-31 декабря - резерв.

Сначала общий контракт

Файл `backend/renderer/models.py`, владелец A2; изменения согласуют A1 и B2.

Объект	Что содержит	Как использовать
`LyricLine`	`id`, `text`, `start_ms`, `end_ms`	Время только целыми мс, null сюда не передавать
`RenderSpec`	`style`, `duration_ms`, `lines: tuple[LyricLine, ...]`	Неизменяемый снимок одной генерации
`RenderResult`	`path`, `duration_ms`, `width`, `height`, `elapsed_seconds`	Worker сохраняет итог, API строит URL отдельно
`MediaInfo` в `ffmpeg.py`	длительность, кодеки, ширина/высота	Результат ffprobe, а не доверие расширению файла

B1 хранит незаполненные тайминги в черновике как null. B2 перед запуском проверяет готовность и создаёт отдельный снимок; редактирование проекта не меняет текущий job. Renderer не импортирует FastAPI, SQLAlchemy или Redis. В API нельзя отдавать внутренний `path`.

A2.01. `RenderValidationError.__init__` и `validate_spec`

Файл: `backend/renderer/validation.py`. **Статус:** реализованы.

- **Вход/выход:** `RenderSpec -> None`; ошибка `RenderValidationError(field, message)`.

- **Что сделать:** проверить действующие правила вместе с B1/F2, расширять только по согласованному контракту. Сейчас: стиль brat, 5000..60000 мс, 1..100 строк, непустой текст до 120 символов, непустые уникальные ID до 64 символов, целые времена, минимум 100 мс, порядок и отсутствие пересечений. Смежные строки и паузы разрешены.
- **Проверка:** `backend/tests/test_renderer_contract.py`; дополнить сравнением одинаковых invalid fixtures с API и UI. `bool`/float/null не являются целыми таймингами.
- **Зависимости:** согласование контракта; не ждёт FFmpeg.
- **Передать:** Руслане для API, Леониду для UI, Андрею для рендера. Андрей и Руслана независимо запускают проверки Татьяны до merge.

A2.02. `generate_tone` и `main`

Файл: `tests/fixtures/generate_tone.py`. **Статус:** реализованы.

- **Вход/выход:** `generate_tone(Path, duration_seconds=10) -> None`; CLI принимает путь и необязательный `--seconds`, пишет новый mono PCM WAV 16 kHz/16 bit.
- **Что сделать:** использовать генератор для 5, 10, 60 секунд и граничных тестов. Сохранить отказ от перезаписи файла. JSON `render-spec.json` соответствует 10 секундам.
- **Проверка:** Python `wave` подтверждает число каналов, частоту и frames; повторный запуск на тот же путь отклоняется. Команда есть в `tests/fixtures/README.md`.
- **Зависимости:** только Python; не нужен чужой трек или сеть.
- **Передать:** Андрею для CLI, Варваре для CI, Руслане для upload. Тон не проверяет вокал и ASR: отдельную разрешённую голосовую запись подготовить с автором.

A2.03. `probe_media`

Файл: `backend/renderer/ffmpeg.py`. **Статус:** TODO.

- **Вход/выход:** `probe_media(path: Path, timeout_seconds=15) -> MediaInfo`.
- **Что написать:** список аргументов ffprobe, `shell=False`, JSON-вывод, ограничение времени и обработку ненулевого exit code. Найти нужные потоки, преобразовать длительность в целые мс, отличать отсутствие аудио от повреждения. Не возвращать технический stderr целиком в пользовательский интерфейс.
- **Проверка:** WAV, MP3, MP4, пустой/повреждённый файл, нет аудио, timeout; сравнить длительность синтетического WAV с известным числом samples.
- **Зависимости:** установленный ffprobe; B2 добавляет его в backend image.
- **Передать:** Андрею для контроля MP4 и Руслане для upload; Андрей ревьюит обработку процессов, Руслана - форму ошибки для API.

A2.04. `validate_audio`

Файл: `backend/renderer/ffmpeg.py`. **Статус:** TODO.

- **Вход/выход:** `validate_audio(path: Path) -> MediaInfo` либо понятная ошибка.
- **Что написать:** проверить размер до ffprobe, реальный контейнер MP3/WAV, наличие звукового потока и длительность 5000..60000 мс. MIME и расширению не доверять. При необходимости расширить `MediaInfo` полем контейнера согласованным PR.

- **Проверка:** 20 MiB и больше; 4999/5000/60000/60001 мс с учётом реального округления ffprobe; переименованный текстовый файл; WAV с неверным заголовком.
- **Зависимости:** `probe_media`; B1 ограничивает байты ещё во время upload, иначе после полной загрузки уже поздно защищать память/диск.
- **Передать:** Руслане для `app.services.audio.store_audio`; API отображает 413/415/422 по согласованным причинам. Руслана принимает интеграцию.

A1.01. `format_timestamp`

Файл: `backend/renderer/ass.py`. **Статус:** TODO.

- **Вход/выход:** `format_timestamp(milliseconds: int) -> str`, формат `H:MM:SS.cc`.
- **Что написать:** выбрать и документировать округление миллисекунд до сотых; одинаковая граница соседних строк должна сериализоваться одинаково.
- **Проверка:** 0, 99, 100, 999, 1000, 59999, 60000; соседние строки, минимальный интервал и перенос через минуту; без float-накопления ошибки.
- **Зависимости:** правила интервалов от A2.
- **Передать:** Татьяне для проверки, затем использовать в `build_ass`.

A1.02. `escape_text`

Файл: `backend/renderer/ass.py`. **Статус:** TODO.

- **Вход/выход:** `escape_text(text: str) -> str` для обычного текста ASS.
- **Что написать:** безопасный сериализатор; скобки и обратные слеши из ввода не должны включать команды форматирования. Согласовать поведение переносов строки.
- **Проверка:** кириллица, кавычки, фигурные скобки, `\\`, `\\N`, текст с последовательностью ASS-тега. Проверить не только строку ASS, но и реальный кадр.
- **Зависимости:** шрифт с кириллицей; не требует API.
- **Передать:** Татьяне для набора враждебных входов; использовать в `build_ass`.

A1.03. `wrap_text`

Файл: `backend/renderer/ass.py`. **Статус:** TODO.

- **Вход/выход:** `wrap_text(text, max_width_px, font_size) -> str`.
- **Что написать:** перенос по измеренной ширине выбранного шрифта; ограничить безопасную область кадра. Если текст не помещается даже по принятому правилу, вернуть ошибку, не обрезать молча. Измеритель/зависимость согласовать с B2.
- **Проверка:** короткая строка, 120 букв, длинное слово без пробелов, кириллица и цифры; визуально сравнить с 1280x720.
- **Зависимости:** фиксированные шрифт, размер, поля; `escape_text`.
- **Передать:** A2 для визуального QA, F2 сообщить ограничение редактирования.

A1.04. `build_ass`

Файл: `backend/renderer/ass.py`. **Статус:** TODO.

- **Вход/выход:** `build_ass(spec: RenderSpec) -> str`, полный ASS UTF-8.
- **Что написать:** проверить снимок; заголовок, разрешение, стиль и Dialogue для каждой строки; чёрный текст по центру. Пропуски времени оставлять пустыми. Шрифт и безопасные поля закрепить в одном месте, не дублировать константы.
- **Проверка:** соседние строки, пауза, первая строка с 0, последняя до конца; проверить показ через FFmpeg после готовности `build_command`.
- **Зависимости:** `validate_spec`, `format_timestamp`, `escape_text`, `wrap_text`.
- **Передать:** Татьяне пример ASS и изображение кадра, затем `render_video`.

A1.05. `build_command`

Файл: `backend/renderer/ffmpeg.py`. **Статус:** TODO.

- **Вход/выход:** `build_command(spec, audio: Path, subtitles: Path, output: Path) -> list[str]`.
- **Что написать:** аргументы для цвета #8ace00, libass, 1280x720, 30 fps, libx264/AAC/yuv420p/faststart и длительности снимка. Субтитры передавать отдельным файлом с безопасным сгенерированным именем. Проверить пробелы и Windows-пути.
- **Проверка:** argv является списком; пользовательский текст не попадает в shell или фильтр; реальный короткий рендер подтверждает нужные потоки и формат.
- **Зависимости:** согласованный Docker image с ffmpeg/libass/кодеками/шрифтом; `build_ass`. Основное целевое окружение worker - Linux Docker.
- **Передать:** B2 для воспроизводимого образа, A2 для ffprobe-проверки.

A1.06. `run_ffmpeg`

Файл: `backend/renderer/ffmpeg.py`. **Статус:** TODO.

- **Вход/выход:** `run_ffmpeg(arguments: list[str], timeout_seconds=240) -> None`; ненулевой exit code/timeout становятся исключениями.
- **Что написать:** контролируемый subprocess с `shell=False`, timeout, ограниченным логом, завершением процесса при сбое. Не скрывать ошибки кодека. RQ timeout 300 с должен быть больше внутреннего timeout FFmpeg.
- **Проверка:** успешный процесс, неверные аргументы, искусственный timeout; после timeout процесс не остаётся, итоговый MP4 не опубликован.
- **Зависимости:** `build_command`; договор ошибок/timeout с B2.
- **Передать:** Варваре сценарии для `run_render_job`; Татьяне инструкцию воспроизведения на отдельном тестовом окружении.

A1.07. `render_video`

Файл: `backend/renderer/render.py`. **Статус:** TODO после проверки снимка.

- **Вход/выход:** `render_video(spec, audio_path: Path, output_path: Path) -> RenderResult`.

- **Что написать:** проверить снимок и аудио, создать ASS и временный MP4 в каталоге job, запустить FFmpeg, проверить готовый файл через ffprobe, атомарно опубликовать `output_path`. При сбое удалить только свои временные файлы. Замерять время; не держать БД-транзакцию и не обновлять HTTP/RQ-статус здесь.
- **Проверка:** настоящий MP4 10 с с кириллицей/звуком; H.264/AAC, 1280x720, отклонение длительности <=200 мс, строки <=100 мс от эталона; broken input и timeout. Следующий тест B2 проверяет, что правка draft не меняет уже созданный снимок.
- **Зависимости:** все предыдущие A1-функции, `probe_media`, `validate_audio`.
- **Передать:** B2 готовую функцию + входной JSON + команду CLI + результат проверки. Приёмка: сначала CLI до 20 ноября, затем настоящий браузерный MP4 до 27 ноября.

A1.08. `main` CLI

Файл: `backend/renderer/cli.py`. **Статус:** вход реализован.

- **Вход/выход:** `python -m renderer.cli spec.json audio.wav output.mp4`; сейчас код 3 для незавершённого рендера, 2 для неверного входа, после реализации 0.
- **Что сделать:** сохранить внешний контракт, дополнить обработку согласованных ошибок FFmpeg после реализации; не печатать успех при отсутствующем MP4.
- **Проверка:** valid JSON + fixture, invalid-overlap, нет аудио, повреждённый JSON; до реализации ожидаемый код 3 не считается тестом генерации.
- **Зависимости:** `render_video`, JSON fixtures.
- **Передать:** Варваре для воспроизведения worker-ошибок и Татьяне для ручного прогона.

После v0, не брать в декабрьскую приёмку

`backend/renderer/future/transcription.py::transcribe_audio(Path, language="ru") -> str`: A1 позже сравнит готовые модели на разрешённом вокале, проверит ресурсы и условия использования; результат всегда можно поправить. До исследования функция остаётся TODO.

`backend/renderer/future/alignment.py::align_lyrics(Path, list[str]) -> tuple[LyricLine, ...]`: A2 позже подготовит ручной эталон и сравнит ошибки таймингов, повторы и паузы. Передать F2 редактируемый результат только после согласования нового API с B1. Эти функции не должны задерживать ручной сценарий и не входят в зависимости v0.

QA.01. Подготовить входы и правила приёмки

Когда: параллельно A2.01..04. **Владелец:** Татьяна, ревью Андрей + Руслана. Зафиксировать `docs/test-plan.md`, собрать валидный WAV/MP3, неправильные файлы, краевые JSON и собственную короткую вокальную запись с разрешением. Для каждой ошибки указать поле/API-код/текст UI. Создавать большие fixtures программно. Готово: любой участник воспроизводит вход из README, источники указаны в LICENSE.md.

QA.02. Принять первую интеграцию

Когда: 21-27 ноября. **Пары:** Татьяна+Илья, Андрей+Варвара. На одном SHA загрузить файл через UI, ввести три строки, сохранить, запустить, дождаться succeeded, просмотреть и скачать MP4. Записать SHA, окружение, `job_id`, команду ffprobe и результаты. Mock-ответ/заранее готовое видео не принимаются. Если сценарий сломан, issue получает P0 и конкретного владельца.

QA.03. Добавить автоматический сквозной сценарий

Когда: после первой рабочей интеграции. **Пары:** Татьяна+Илья, помощь Варвары. Завести тест в `tests/e2e/` (будущая реализация): загрузка fixture, ввод таймингов, генерация, скачивание. Использовать отдельную БД/каталог и настоящий worker/renderer. Разработчики продолжают unit/API-тесты своих функций. E2E-check становится обязательным в GitHub только после реального стабильного запуска; отсутствующий тест не обозначать зелёным.

QA.04. Устойчивость и независимый ретест

Когда: до 6 декабря ручной сценарий; 7-20 декабря регрессия. Проверить сохранение после F5, повтор failed, двойной клик, потерю сети, рестарт, Range, чужую сессию, длинные строки, два браузера и второе устройство. Результат писать в `docs/test-report.md`, дефект - в issue со шагами/ожиданием/фактом. Автор исправляет, другой участник делает ретест на новом SHA. Алгоритм Татьяны перепроверяет Андрей, её интеграцию с upload - Руслана; общие UI-сценарии - Леонид.

QA.05. Сдать проверяемый релиз

Когда: 21-27 декабря. **Владельцы:** Татьяна + Варвара, проверяет Руслана. Повторить чистый запуск по README на итоговом SHA, пройти матрицу, закрыть P0 и P1 основного сценария, указать оставшиеся ограничения. Приложить test-report, вклад участников, 3-5-минутный demo-сценарий. Варвара ставит тег и публикует Release после review; Татьяна сверяет SHA стенда и отчёта. Незапущенные проверки остаются NOT RUN.

GitHub: первая настройка и работа каждого участника

Репозиторий уже существует: https://github.com/rainingrsm/rztlyrics. Локально есть `main`, remote `origin`, один начальный коммит. Эти сведения проверены при подготовке каркаса; перед выполнением команд снова смотрите `git status` и `git branch -a`. Настройки GitHub, роли и PR этим каркасом не созданы. Ниже действия команды. Команды рассчитаны на PowerShell из корня репозитория, кроме явно указанного `cd`.

1. Владелец репозитория + Варвара: загрузить каркас один раз

В существующей папке **не выполняйте git init и не создавайте второй origin**. Сначала настройте своё имя и подтверждённый GitHub email (можно noreply из Settings → Emails).

```
cd C:\Users\Windows\Desktop\rztlyrics
git config user.name "Ваше имя"
git config user.email "Ваш подтверждённый email"
git status
git remote -v
git fetch origin
git switch -c codex/bootstrap-scaffold
git add .dockerignore .editorconfig .env.example .gitattributes .gitignore .nvmrc .python-version
git add README.MD CONTRIBUTING.md LICENSE.md backend frontend docs
git add infra compose.yaml .github tests scripts output/pdf
git diff --staged --stat
git diff --staged
git commit -m "chore: add RZT project scaffold and team plan"
git push -u origin codex/bootstrap-scaffold
```

Перед commit проверьте, что нет `.env`, аудио, node_modules, .venv и tmp. Файлы документации и итоговый PDF включены намеренно. При ошибке команды остановитесь и прочитайте её; не используйте `--force` или `reset --hard`.

На GitHub → Pull requests → New pull request: **base main**, compare `codex/bootstrap-scaffold`. Это единственное стартовое исключение: `develop` ещё не создана. Назначьте ревьюера Руслану или другого участника, кроме автора. Дождитесь frontend/backend checks. Merge через **Create a merge commit**. Если правила уже включены, выполните их, не обходите. После merge, с чистой рабочей папкой:

```
git switch main
git pull --ff-only origin main
git switch -c develop
git push -u origin develop
```

Если `develop` уже существует, не создавайте заново: `git fetch origin`, затем `git switch develop` (либо `git switch --track origin/develop`, если нет локальной).

2. Владелец: доступ, защита, доска

1. Settings → Collaborators: пригласите остальных пятерых по настоящим GitHub-логинам. Всем нужен доступ на запись; административные права не требуются для обычных PR.
2. Settings → General: default branch `main`; разрешите merge commits. Не используйте squash/rebase merge в процессе команды. Автоудаление head-веток выключите, чтобы после релиза не удалить `develop`.

3. Settings → Branches → Add classic branch protection rule: отдельно `main`, отдельно `develop`. Требовать PR, 1 approval, решение обсуждений; сбрасывать approval после новых изменений. Запретить force push/удаление. Применить правило и к администраторам при доступности настройки. Не включать linear history: команда использует merge commits.
4. После первого успешного Actions запуска добавьте фактические checks `frontend` и `backend` в required status checks. Требуйте актуальность ветки. Пока не добавляйте renderer-e2e: такого исполняемого check ещё нет. Нельзя делать обязательным несуществующий check.
5. В `.github/CODEOWNERS` замените закомментированные примеры реальными логинами с write. До этого не включайте обязательный Code Owners review. Это отдельный маленький PR Варвары.
6. Создайте milestone `v0.0.1`, ориентир 27.12.2026, labels `area:frontend`, `area:backend`, `area:algorithm`, `area:qa`, `area:docs`, `priority:P0`, `priority:P1`, `priority:P2`.
7. Доска Project: Backlog → Ready → In progress → Review → QA → Done. Заведите задачи из docs/backlog.md; каждый пункт подробного плана делите на PR до 1-2 дней.

Защита веток зависит от тарифа/видимости репозитория. Если настройка недоступна, проверьте применимость своего тарифа, не отмечайте защиту включённой без проверки. Публичность и требования организаторов команда подтверждает отдельно. Официальная документация защиты.

3. Каждый участник: получить репозиторий на свой компьютер

Если папки ещё нет:

```
git clone https://github.com/rainingrsm/rztlyrics.git
cd rztlyrics
git config user.name "Ваше имя"
git config user.email "Ваш подтверждённый email"
git fetch origin
git switch --track origin/develop
```

Если локальная develop уже есть: `git switch develop`, `git pull --ff-only origin develop`. Для входа используйте GitHub Desktop/менеджер учётных данных или настроенный SSH. Пароль аккаунта и токен не вставляйте в исходники или URL remote.

4. Первая ветка Ильи (F1)

Возьмите F1-01 в backlog, назначьте себе. Читайте docs/tasks/frontend.md, раздел F1.

```
git switch develop
git pull --ff-only origin develop
git switch -c codex/f1-01-upload
cd frontend
npm ci
npm run dev
# После правок остановите dev Ctrl+C и выполните:
npm run lint
npm run test -- --run
npm run build
cd ..
git add frontend/src/features/upload frontend/src/shared/api
git diff --staged
git commit -m "feat(upload): connect project creation and audio upload"
git push -u origin codex/f1-01-upload
```

Добавляйте только реально изменённые файлы задачи. В PR base `develop`, ревьюер Леонид; Руслана проверяет HTTP-контракт. Следующие ветки: `codex/f1-02-results`, `codex/f1-03-ui-errors`.

5. Первая ветка Леонида (F2)

Задача F2-01, подробности в docs/tasks/frontend.md.

```
git switch develop
git pull --ff-only origin develop
git switch -c codex/f2-01-editor
cd frontend
npm ci
npm run dev
# После правок:
npm run lint
npm run test -- --run
npm run build
cd ..
git add frontend/src/features/editor
git diff --staged
git commit -m "feat(editor): edit and validate manual line timings"
git push -u origin codex/f2-01-editor
```

PR в `develop`, ревьюер Илья; Татьяна смотрит интервалы. Следующие ветки: `codex/f2-02-save-draft`, `codex/f2-03-save-render-flow`. Не меняйте API-клиент параллельно с Ильёй без короткого согласования.

6. Первая ветка Русланы (B1)

Задача B1-01, docs/tasks/backend.md. Сначала сессия/проект, затем upload и draft.

```
git switch develop
git pull --ff-only origin develop
git switch -c codex/b1-01-projects
cd backend
uv sync --frozen
uv run alembic upgrade head
uv run uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
# После правок и остановки сервера:
uv run ruff check .
uv run ruff format --check .
uv run pytest
cd ..
git add backend/app/services/sessions.py backend/app/services/projects.py backend/app/api/projects.py
git diff --staged
git commit -m "feat(projects): persist projects with anonymous ownership"
git push -u origin codex/b1-01-projects
```

Если менялись модели/миграции/тесты - добавьте их отдельным `git add` и проверьте diff. Ревьюер Варвара; Илья проверяет вызов createProject. Далее `codex/b1-02-upload`, `codex/b1-03-lyrics`. Не добавляйте созданную SQLite БД.

7. Первая ветка Варвары (B2)

Задача B2-01: воспроизводимый Compose/CI; B2-02: lifecycle jobs.

```
git switch develop
git pull --ff-only origin develop
git switch -c codex/b2-01-integration
Copy-Item .env.example .env
docker compose config
docker compose build
docker compose run --rm api uv run --no-sync alembic upgrade head
docker compose up -d
docker compose ps
docker compose logs --tail=50 api worker
git add compose.yaml infra .github
git diff --staged
git commit -m "chore(infra): verify shared API and worker environment"
git push -u origin codex/b2-01-integration
```

`Copy-Item` нужен один раз; существующий `.env` не перезаписывайте. Ревьюер Руслана; Андрей проверяет FFmpeg внутри worker. Далее `codex/b2-02-jobs`, `codex/b2-03-video-delivery`, `codex/b2-04-staging`. Windows worker - в Linux-контейнере. Lock-файлы меняйте штатным npm/uv, не вручную.

8. Первая ветка Андрея (A1)

Задача A1-01: ASS/CLI, затем настоящий MP4. Подробности docs/tasks/algorithm-qa.md.

```
git switch develop
git pull --ff-only origin develop
git switch -c codex/a1-01-ass
cd backend
uv sync --frozen
uv run ruff check .
uv run ruff format --check .
uv run pytest tests/test_renderer_contract.py
cd ..
git add backend/renderer/ass.py backend/tests/test_renderer_contract.py
git diff --staged
git commit -m "feat(renderer): serialize safe ASS subtitles"
git push -u origin codex/a1-01-ass
```

Ревьюер Татьяна; Варвара принимает функцию render_video после следующей задачи. Далее `codex/a1-02-ffmpeg`, `codex/a1-03-render-pipeline`. MP4 и пользовательское аудио в Git не добавляйте. Проверку выполненного рендера записывайте в PR: команда, длительность, кодеки, параметры, время исполнения.

9. Первая ветка Татьяны (A2 + QA)

Задача A2-01: граничные проверки/fixtures. Затем probe, потом QA основного сценария.

```
git switch develop
git pull --ff-only origin develop
git switch -c codex/a2-01-validation
cd backend
uv sync --frozen
uv run pytest tests/test_renderer_contract.py tests/test_schemas.py
uv run ruff check .
uv run ruff format --check .
cd ..
git add backend/renderer/validation.py backend/tests/test_renderer_contract.py tests/fixtures
git diff --staged
git commit -m "test(renderer): cover timing boundaries and invalid inputs"
git push -u origin codex/a2-01-validation
```

Ревьюер Андрей. Далее `codex/a2-02-probe`, `codex/qa-01-smoke`, `codex/qa-02-regression`. Для QA-ветки коммитьте docs/test-report.md, сценарии и автоматизацию, не только скриншоты. Дефекты заводите через bug template с SHA и шагами. Самостоятельно не принимайте свой PR: Андрей проверяет алгоритм, Руслана/Леонид - интеграцию.

10. Для всех: что делать после push

1. Pull requests → New: **base develop**, compare ваша ветка. Draft, если ещё не готово.
2. Заголовок описывает поведение. Тело по шаблону: `Refs #реальный_номер`, что изменилось, команды проверки, ожидаемый результат, ограничения. Номера из примеров не копируйте.
3. Назначьте указанного ревьюера. Изменение общего DTO согласуют обе стороны.
4. Замечания исправляйте коммитами в этой же ветке; `git push` обновит PR.
5. После approval, зелёного CI, разрешённых обсуждений - merge commit в develop.
6. Татьяна проверяет общий SHA и вручную закрывает issue с результатом. Для её задач это делает независимый проверяющий. `Closes #N` в PR в develop автоматически issue не закрывает при default main: правило GitHub.

Обновление своей ветки перед review (сначала сохраните собственные изменения коммитом):

```
git status
git fetch origin
git merge origin/develop
# При конфликте согласуйте результат с владельцем файла, затем:
git add путь-к-исправленному-файлу
git commit
# Запустите проверки своей части
git push
```

`git commit` после merge нужен только если были конфликты, разрешённые вручную. Если merge без конфликтов уже создал коммит, повторный commit не требуется. Если не понимаете конфликт, `git merge --abort` отменяет начатое слияние, затем позовите коллегу.

После успешного merge и проверки:

```
git switch develop
git pull --ff-only origin develop
git branch -d codex/имя-вашей-слитой-ветки
git fetch --prune
```

Удалите удалённую **ветку задачи** кнопкой Delete branch в PR. `main`/`develop` сохраняйте. Следующую задачу всегда начинайте новой веткой из свежей develop.

11. Варвара + Руслана: релиз в декабре

PR **develop** → **main**, проверка конечного SHA, согласование QA. После merge:

```
git switch main
git pull --ff-only origin main
git rev-parse HEAD
git tag -a v0.0.1 -m "RZT manual lyrics video v0.0.1"
git push origin v0.0.1
```

Тег ставьте только на проверенный коммит. Если тег уже существует, не перемещайте его: исправление оформляется новой версией. GitHub → Releases → Draft a new release → выберите тег; добавьте SHA, URL стенда, инструкции, ограничения, тест-отчёт и демо. Проверьте ссылки с другого устройства, затем публикуйте. Официальная инструкция Release.

План приёмки v0

Владелец: Татьяна (A2 + QA). Каждый разработчик проверяет свои функции; приёмка кода Татьяны - Андрей/Руслана. Это план будущих продуктовых проверок, не отчёт о готовности. Результаты и SHA записываются в `test-report.md`.

Среда и правила

- Отдельное тестовое окружение: свои volume/БД/каталог; не ломать демонстрационный стенд.
- Исходный fixture: `tests/fixtures/render-spec.json` и WAV из `generate_tone.py`. MP3 и голосовой пример с подтверждённым источником команда добавит отдельно.
- На каждом прогоне: SHA, URL/команда запуска, ОС, браузер/версия, дата, исполнитель.
- PASS только после наблюдаемого результата; FAIL со ссылкой на issue; NOT RUN если сценарий не запускали; BLOCKED с причиной, если отсутствует зависимость.
- P0: основной сценарий не работает/теряет данные; P1: обязательная функция неверна; P2: оформление/необязательное удобство. В релизе нет открытых P0/P1 основного сценария.

ID	Действие / вход	Ожидаемый результат	Исполнитель и поддержка
T01	Загрузить MP3/WAV 5-60 с	Файл принят, длительность серверная, звук в плеере	A2 + F1/B1
T02	Пустой, повреждённый и переименованный текстовый файл	Понятная ошибка, нет зависшего upload/остатка	A2 + B1
T03	20 MiB/больше; 5/60 с и за границами	Сервер применяет объявленные лимиты, UI согласован	A2 + B1
T04	Кириллица, кавычки, скобки, обратный слеш, ASS-последовательность	Обычный текст, команды ASS не выполняются	A2 + A1
T05	0/1/100/101 строка; 120/121 символ	Draft может быть пустым; render требует 1-100 валидных строк	A2 + F2/B1
T06	Сохранить null; отрицательные, float, конец до начала	null допустим в draft; неверные данные не запускают render	A2 + F2/B1
T07	Пересечение, обратный порядок, интервал <100 мс, конец за аудио	Ошибка нужного поля, renderer не запускается	A2 + A1/B1
T08	Смежные строки и пауза из fixture	В один момент одна строка; в паузе только фон	A2 + A1
T09	Сохранить, F5, повторно открыть URL той же сессией	Черновик/аудио восстановлены	A2 + F2/B1

ID	Действие / вход	Ожидаемый результат	Исполнитель и поддержка
T10	Двойной быстрый запуск одного проекта	Только одна активная задача, понятный ответ 409	A2 + B2/F1
T11	Запустить два проекта подряд	Очередь работает, результаты не перепутаны	A2 + B2
T12	Потерять сеть во время polling, восстановить	Опрос продолжается для того же job, новый render не создан	A2 + F1
T13	Контролируемый exit code/timeout renderer	failed, процесс завершён, неполный MP4 скрыт, повтор возможен	A2 + A1/B2
T14	Перезапустить API/worker/стенд	Сохранённые данные остаются; потерянные jobs объяснимо failed	A2 + B1/B2
T15	Просмотр, перемотка, download=1, открыть скачанный MP4	Звук/видео есть, Range работает, корректный Content-Disposition	A2 + F1/B2
T16	Сравнить начала строк/длину с эталоном	Время строки ±100 мс; длина ±200 мс; метод измерения записан	A2 + A1
T17	60 с, 100 строк, длинные строки	Текст не обрезан; время/память измерены; цель рендера ≤3 мин	A2 + A1/B2
T18	Чужая сессия, неизвестный UUID, путь в имени файла	404/отказ; нет доступа к чужим файлам и выхода из data	A2 + B1/B2
T19	Chromium/Firefox и второе устройство	Полный сценарий проходит; точные версии записаны	A2 + F1/F2
T20	Чистый запуск по README и внешние ссылки	Команды воспроизводимы; репозиторий/Release/стенд доступны	A2 + B2
T21	После render изменить текст/тайминги проекта	Текущий job использует свой snapshot; старый результат помечен	A2 + F2/B2
T22	Redis недоступен; очередь полна	503/429 по контракту, не создана неотслеживаемая задача	A2 + B2
T23	Очистка истёкших данных при активном job	Активные jobs не удалены; старые данные очищены по политике	A2 + B1/B2

Измерение и доказательства

Для T16 нужны эталонные времена и извлечённые кадры около границ, а не ощущение «вроде совпало». 30 fps ограничивает разрешение кадрового сравнения. A1+A2 заранее договариваются о методе и сохраняют измеренные значения в отчёте. Для T17 указать CPU/RAM, настройки, длину входа, wall time и пиковую память; цель ≤3 минут не объявлять достигнутой без замера на выбранном стенде.

Минимальный автоматический набор после реализации: unit-интервалы/ASS, API доступ/ошибки/двойной job, настоящий короткий FFmpeg-рендер и один UI→MP4 E2E. Текущий CI каркаса не является этой полной приёмкой.

Контрольные точки: 27 ноября T01+T08+T15 через сайт; 6 декабря законченный ручной сценарий; 20 декабря release candidate; 27 декабря повтор на релизном SHA.

Демонстрация после реализации v0

Сейчас каркас этот сценарий не выполняет. Владелец сценария - Татьяна, показывают Илья/Леонид. Используйте собственную короткую вокальную запись; технический tone.wav годится только для smoke.

1. Показать URL стенда, версию и SHA. Объяснить задачу сервиса за 20 секунд.
2. Загрузить MP3/WAV 10-20 секунд. Показать проверенную длительность и плеер.
3. Ввести 3 короткие собственные строки; задать времена, одну границу взять из плеера.
4. Нарочно задать неверный интервал, увидеть ошибку, исправить.
5. Сохранить и обновить страницу: данные остаются.
6. Запустить рендер; показать queued/running. Заранее измерить ожидание на стенде.
7. Просмотреть MP4 со звуком, скачать, открыть локально.
8. Показать GitHub Release, командный вклад и ограничения v0.

Если стенд недоступен, воспроизвести на локальном Compose. Записанный ролик можно приложить как демонстрационный материал, но нельзя выдавать за текущую работу сервиса.

Проверка стартового каркаса

Редакция 9 ноября 2026. Это проверка структуры и реализованных утилит, не приёмка продукта.

Проверка	Результат
npm ci	PASS, установка из package-lock.json
npm run lint	PASS
npm run test -- --run	PASS, 25 тестов
npm run build	PASS
pytest backend/tests	PASS, 40 тестов
Ruff check + format --check	PASS
Alembic upgrade head + check	PASS на отдельной временной SQLite БД
Генератор WAV и отказ перезаписи	PASS
Renderer CLI	Ожидаемый код 3, MP4 не создаётся
Docker Compose / FFmpeg в Linux-контейнере	NOT RUN: Docker отсутствует в среде подготовки
GitHub Actions	NOT RUN: коммиты и push не выполнялись
Полный браузер → MP4	NOT IMPLEMENTED

Среда: Windows, Python 3.12.14, локальный Node 26.13.0. Для команды/CI закреплена ветка Node 22, минимум 22.13; прогон на этой ветке проверит первый CI. Один warning Starlette об устаревающем httpx TestClient не приводит к падению тестов; обновление тестового клиента - отдельная задача при обновлении зависимостей.

Доказательства готовности будущего v0 записываются в docs/test-report.md с реальным SHA, устройствами, стендом и результатами. Не переносите эти PASS на продуктовые сценарии.

Основание и ссылки

Использованы предоставленные lyrics_video_v0_team_plan (2).pdf, rztp (1).pdf и шесть изображений аннотации. Два последних изображения дублируют раздел дальнейших идей. Фрагменты, которых нет в приложениях, не восстанавливались как требования организаторов.

Текущие уточнения команды: Python backend; Илья/Леонид frontend; остальные распределены между backend и алгоритмом; QA назначена по нагрузке; срок - конец декабря. Технические настройки, часовые оценки и дата внутреннего релиза являются предложениями этого плана.

GitHub проекта

Защита веток GitHub

PR и issues

GitHub Releases

uv Docker · RQ workers · Vite

Полные команды запуска находятся в README.MD. Публичный стенд, настройки GitHub, release и продуктовая приёмка не выполнялись автоматически при создании каркаса.